

Universitatea Națională de Știință și Tehnologie POLITEHNICA
București

Facultatea de Automatică și Calculatoare

Departamentul de Calculatoare



LUCRARE DE DIPLOMĂ

**CarValuator: aplicație web pentru evaluarea
prețurilor autoturismelor second-hand folosind
web scraping și modele de învățare automată**

Conducător științific:

Popescu Cornel

Autor:

Nicula Dan-Alexandru

București, 2026

SINOPSIS

CarValuator este o aplicație web destinată evaluării preliminare a anunțurilor pentru autoturisme second-hand. Utilizatorul introduce un link Autovit sau mobile.de, iar sistemul extrage caracteristicile vehiculului, normalizează datele și estimează prețul cu ajutorul mai multor modele de regresie. Predicțiile sunt combinate într-o estimare finală ponderată, pe baza căreia este afișat un verdict ușor de interpretat: preț corect, prea mic sau prea mare.

Lucrarea prezintă colectarea datelor prin web scraping, deduplicarea și pregătirea setului de antrenare, selecția caracteristicilor și compararea algoritmilor SVR, Ridge, KNN, Random Forest, Extra Trees, Gradient Boosting și Voting Regresor. Aplicația include autentificare, istoric individual, toleranță configurabilă, explicații pentru metricile utilizate și recomandări de anunțuri similare. Rezultatele experimentale indică un scor R^2 maxim de aproximativ 0.923 și o eroare absolută medie minimă de aproximativ 2696 EUR. Principalele limitări sunt dimensiunea și distribuția datasetului, dependența de structura site-urilor externe și lipsa unor informații tehnice complete.

ABSTRACT

CarValuator is a web application designed for the preliminary evaluation of second-hand car advertisements. The user submits an Autovit or mobile.de link, and the system extracts the vehicle characteristics, normalizes the data, and estimates its price using several regression models. Individual predictions are combined into a weighted final estimate, which produces an accessible verdict: fair price, suspiciously low price, or excessive price.

The thesis presents the complete workflow, including web scraping, deduplication, training-data preparation, feature selection, and the comparison of SVR, Ridge, KNN, Random Forest, Extra Trees, Gradient Boosting, and Voting Regressor models. The application provides authentication, account-specific history, configurable verdict tolerance, explanations of the reported metrics, and recommendations for similar advertisements. Experimental results show a maximum R^2 score of approximately 0.923 and a minimum mean absolute error of approximately 2696 EUR. The main limitations are the size and distribution of the dataset, dependence on external website structures, and incomplete availability of relevant technical vehicle information.

Cuprins

Sinopsis / Abstract	i
1 Introducere	1
1.1 Contextul problemei	1
1.2 Motivația proiectului	2
1.3 Obiectivele lucrării	2
1.4 Metodologie generală	3
1.5 Contribuții	3
1.6 Structura lucrării	4
2 Analiza și specificarea cerințelor	5
2.1 Profilul utilizatorilor	5
2.2 Nevoi identificate	6
2.3 Cerințe funcționale	6
2.4 Cerințe nefuncționale	7
2.5 Criterii de acceptare	8
3 Studiu de piață și fundamentare	9
3.1 Evoluția pieței second-hand	9
3.2 Autovit	9
3.3 mobile.de	10
3.4 Poziționarea soluției propuse	10
3.5 Riscuri și limite ale comparației	11
3.6 Fundamentare teoretică și lucrări conexe	12
3.6.1 Evaluarea prețurilor ca problemă de regresie	12
3.6.2 Modele folosite	12
3.6.3 Transformarea logaritmică a prețului	13
3.6.4 Selecția caracteristicilor	13
3.6.5 Metrice de evaluare	14
3.6.6 Web scraping și considerații etice	14
3.6.7 Context tehnologic și alegerea platformei de deployment	14
4 Proiectarea aplicației	16
4.1 Obiective de proiectare	16
4.2 Arhitectura modulară	16
4.3 Cazuri de utilizare	17
4.4 Fluxul de execuție al unei predicții	18
4.5 Proiectarea API-ului	19
4.6 Proiectarea navigării	19
4.7 Proiectarea bazei de date	20
4.8 Proiectarea pentru securitate și defecte	21

4.9	Decizii de deployment	21
5	Implementarea aplicației	23
5.1	Colectarea și pregătirea datelor	23
5.1.1	Sursele de date	23
5.1.2	Scraping pentru Autovit	24
5.1.3	Scraping pentru mobile.de	25
5.1.4	Schema datelor brute	26
5.1.5	Normalizare	26
5.1.6	Deduplicare	27
5.1.7	Analiza exploratorie	27
5.1.8	Pregătirea pentru antrenare	29
5.2	Modelare și selecția caracteristicilor	29
5.2.1	Caracteristici derivate	29
5.2.2	Selecția statistică	30
5.2.3	Preprocesare	31
5.2.4	Modele candidate și motivarea alegerii	32
5.2.5	Antrenarea pe log-preț	33
5.2.6	Predicția finală prin medie ponderată	34
5.2.7	Verdictul	34
5.3	Implementarea backend-ului	35
5.4	Implementarea frontend-ului	35
5.5	Implementarea autentificării și securității	37
5.5.1	Limitarea ratei cererilor	38
5.6	Implementarea similarității	38
5.7	Artefacte și deployment pe Render	39
6	Testarea aplicației și rezultate	40
6.1	Rolul studiilor de caz	40
6.2	Scenariu Autovit cu mașină comună	40
6.3	Scenariu Autovit cu mașină premium	41
6.4	Scenariu mobile.de	41
6.5	Scenariu de istoric pe cont	42
6.6	Scenariu de deployment	42
6.7	Testarea limitării ratei	42
6.8	Validarea mesajelor de eroare	43
6.9	Observații din testare	43
6.10	Concluziile studiilor de caz	44
6.11	Evaluare experimentală	45
6.11.1	Configurația experimentului	45
6.11.2	Rezultatele modelelor	45
6.11.3	Analiza predicțiilor	46
6.11.4	Corelații și interpretabilitate	48
6.11.5	Testare prin aplicație	48
6.11.6	Comparație cu indicatorii platformelor	49
6.11.7	Calitatea setului de date	49
6.11.8	Acuratețe și utilitate practică	50
6.11.9	Reproductibilitate	50
7	Concluzii și direcții viitoare	51
7.1	Dificultăți întâlnite	51
7.2	Limitări	52
7.3	Concluzii	52
7.4	Direcții viitoare	53

7.5	Încheiere	53
A	Comenzi utile pentru reproducerea experimentelor	55
A.1	Pornirea mediului	55
A.2	Colectarea datelor	55
A.3	Pregătirea setului de date	56
A.4	Antrenarea modelelor	56
A.5	Pornirea aplicației	56
A.6	Reîmprospătarea sugestiilor similare	57
A.7	Pregătirea pentru Render	57

Listă de figuri

1.1	Fluxul general al proiectului CarValuator	3
4.1	Arhitectura modulară a sistemului CarValuator	17
4.2	Diagramă UML de secvență pentru analiza unui anunț	18
4.3	Matricea funcționalităților în funcție de răspândirea și frecvența utilizării	20
4.4	Schema relațională pentru utilizatori, sesiuni și istoricul predicțiilor	20
5.1	Componenta setului de date după sursă	24
5.2	Distribuția prețurilor în setul de date	27
5.3	Prețul în funcție de kilometraj	28
5.4	Mediana prețului în funcție de anul mașinii	28
5.5	Cele mai frecvente mărci în setul de date	29
5.6	Caracteristici relevante după testele statistice	31
5.7	Interfața web după finalizarea unei predicții	37
6.1	Comparație de performanță între modele	46
6.2	MAE și R^2 pentru modelele evaluate	46
6.3	Preț real față de preț estimat	47
6.4	Analiza reziduurilor pentru modelul ales	47
6.5	Matrice de corelații pentru caracteristici numerice	48

Listă de tabele

3.1	Comparație între abordări	11
4.1	Rutele proiectate pentru aplicația web	19
5.1	Rezumatul setului combinat înainte de curățarea finală pentru antrenare	24
5.2	Câmpuri extrase din anunțuri	26
5.3	Exemple de caracteristici derivate	30
5.4	Caracteristici selectate în antrenarea finală	31
6.1	Performanța modelelor pe setul de test	45
6.2	Exemple de comparație între estimarea CarValuator și etichetele platformelor	49
7.1	Dificultăți și soluții aplicate	52

Notății și abrevieri

API: interfață de programare a aplicațiilor
CSV: valori separate prin virgulă
JSONL: format JSON pe linii
KNN: metoda celor mai apropiați vecini
MAE: eroare absolută medie
ML: învățare automată
RMSE: rădăcina erorii pătratice medii
SVR: regresie cu vectori suport

Capitolul 1

Introducere

1.1 Contextul problemei

Piața autoturismelor second-hand este una dintre cele mai active piețe online pentru utilizatorii obișnuiți. Decizia de cumpărare este dificilă deoarece prețul unui vehicul nu depinde doar de marcă și model, ci și de vârstă, kilometraj, motorizare, dotări, stare, istoric de service, tip de vânzător, localizare și momentul apariției anunțului. În practică, două mașini care par similare la prima vedere pot avea prețuri foarte diferite, iar un cumpărător fără experiență poate interpreta greșit o ofertă.

Dimensiunea pieței justifică nevoia unui instrument de analiză preliminară. În România, segmentul mașinilor second-hand din import a ajuns în 2024 la 344486 de unități, iar reînmatriculările interne au depășit 710000 de unități, cel mai ridicat nivel din ultimii șase ani [18]. La nivel european, analiza Joint Research Centre arată că mașinile noi reprezintă doar o parte din totalul vânzărilor anuale de autoturisme și că datele despre piața vehiculelor rulate sunt dificil de armonizat între state [23]. Acest context susține abordarea proiectului: colectarea unui snapshot propriu de anunțuri și transformarea lui într-un set de date utilizabil pentru estimare.

O ofertă prea mică poate ascunde probleme tehnice, daune nedeclarate, kilometraj modificat, documente incomplete sau chiar o tentativă de fraudă. O ofertă prea mare poate fi doar rezultatul unei așteptări nerealiste din partea vânzătorului sau al unei prezentări comerciale agresive. Între aceste extreme există un interval de preț considerat rezonabil, iar identificarea lui presupune compararea anunțului cu exemple asemănătoare din piață.

Modelele hedonice de preț, introduse în literatura economică de Rosen, tratează bunurile diferențiate ca pachete de caracteristici, iar prețul observat este rezultatul contribuției acestor caracteristici [19]. În cazul autoturismelor, această idee este naturală: puterea motorului, anul, kilometrajul și marca sunt trăsături care influențează valoarea percepută. Lucrarea de față folosește aceeași intuiție, dar într-o formă aplicată: colectează anunțuri reale, extrage caracteristicile vehiculelor și antrenează modele de regresie pentru estimarea prețului.

1.2 Motivația proiectului

Motivația proiectului CarValuator este una practică. Utilizatorul introduce un link de anunț Autovit sau mobile.de, iar aplicația răspunde cu o estimare de preț corect și cu un verdict simplu: preț corect, preț prea mic sau preț prea mare. În loc să ofere doar un număr, aplicația afișează și estimările fiecărui model, diferența procentuală față de prețul publicat, imaginea mașinii, anunțuri similare și o scurtă istorie a analizelor efectuate de contul curent.

O astfel de soluție nu înlocuiește o inspecție tehnică și nu poate garanta că un vehicul este sigur de cumpărat. Totuși, poate reduce timpul necesar unei analize preliminare. Utilizatorul nu mai compară manual zeci de anunțuri, ci primește rapid o măsură orientativă a poziționării prețului. Pentru un proiect de diplomă, această problemă este potrivită deoarece include componente diverse: colectare de date, normalizare, deduplicare, analiză statistică, regresie, interfață web, autentificare și deployment.

1.3 Obiectivele lucrării

Lucrarea are următoarele obiective principale:

- construirea unui pipeline de scraping pentru anunțuri auto publice de pe Autovit și mobile.de;
- salvarea datelor brute într-un format auditabil, JSONL, astfel încât procesarea să poată fi refăcută fără scraping repetat;
- normalizarea câmpurilor relevante pentru învățare automată, inclusiv preț, an, kilometraj, combustibil, transmisie, putere și capacitate cilindrică;
- eliminarea duplicatelor exacte și a duplicatelor probabile folosind chei derivate din caracteristicile vehiculului;
- selecția caracteristicilor semnificative folosind corelație Pearson pentru variabile numerice și teste de tip chi-pătrat cu efect Cramer pentru variabile categorice;
- antrenarea și compararea mai multor modele de regresie, inclusiv SVR, Ridge, KNN și modele bazate pe arbori;
- folosirea unei ținte logaritmice pentru preț, pentru a reduce influența disproporționată a mașinilor foarte scumpe;
- integrarea modelului într-o aplicație web în limba română, cu autentificare, istoric, sugestii similare și controale configurabile pentru verdict.

Aceste obiective urmăresc nu doar obținerea unei predicții, ci și construirea unui sistem coerent, reproductibil și utilizabil. Din acest motiv, lucrarea insistă pe deciziile de proiectare și pe limitele întâlnite, nu doar pe valorile finale ale metricilor.

1.4 Metodologie generală

Fluxul proiectului urmează ideea procesului CRISP-DM, care separă un proiect de data mining în înțelegerea problemei, înțelegerea datelor, pregătirea datelor, modelare, evaluare și implementare [21]. În contextul acestei lucrări, aceste etape au fost adaptate la contextul unei aplicații web.

În prima etapă s-a definit problema de business: utilizatorul vrea să știe dacă prețul unui anunț pare rezonabil. În a doua etapă s-au colectat date din anunțuri publice, cu atenție la limitările tehnice și etice ale scraping-ului. În a treia etapă datele au fost transformate într-un set de antrenare curat. În etapa de modelare au fost comparate mai multe regresii. În etapa de evaluare s-au urmărit RMSE, MAE și R^2 , metrici uzuale pentru regresie [20]. În ultima etapă, cel mai util flux a fost expus prin API și interfață web.

Workflow-ul aplicației este ilustrat în [Figura 1.1](#). Diagrama arată separarea dintre colectarea datelor, pregătirea setului, antrenarea modelelor și utilizarea aplicației.

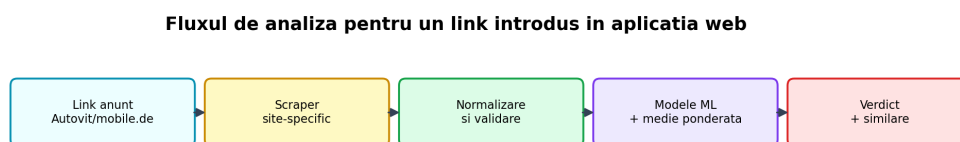


Figura 1.1: Fluxul general al proiectului CarValuator

1.5 Contribuții

Contribuțiile principale ale lucrării sunt următoarele:

- un scraper funcțional pentru Autovit, cu extragere de date din pagini de căutare și pagini de detaliu;
- o integrare pentru mobile.de, cu suport pentru linkuri de detaliu, scraping experimental prin pagini SEO și documentarea problemelor de blocare întâlnite;
- un modul de normalizare care produce câmpuri consecvente pentru modelare;
- un mecanism de deduplicare care elimină atât duplicate exacte, cât și anunțuri foarte asemănătoare;
- un pipeline de antrenare cu selecție statistică a caracteristicilor și modele multiple;
- un mecanism de predicție finală bazat pe medie ponderată a modelelor, cu două strategii selectabile de utilizator;
- o aplicație web completă, cu autentificare, istoric pe cont, afișare de imagini și recomandări similare;

- pregătirea proiectului pentru deployment pe Render folosind variabile de mediu și artefacte compacte.

1.6 Structura lucrării

Capitolul 2 prezintă analiza utilizatorilor și cerințele aplicației. Capitolul 3 discută piața, soluțiile existente pe Autovit și mobile.de, precum și fundamentarea teoretică. Capitolul 4 descrie proiectarea sistemului prin arhitectură modulară, cazuri de utilizare, diagramă de execuție și schema bazei de date. Capitolul 5 explică implementarea scraping-ului, pregătirea datelor, modelele ML, backend-ul, frontend-ul, securitatea și deployment-ul. Capitolul 6 prezintă scenariile de test, rezultatele experimentale și comparația cu indicatorii platformelor. Capitolul 7 sintetizează dificultățile întâlnite, limitele curente și direcțiile viitoare.

Capitolul 2

Analiza și specificarea cerințelor

2.1 Profilul utilizatorilor

Aplicația CarValuator este gândită pentru cumpărători care caută autoturisme second-hand online și au nevoie de o verificare rapidă a prețului. Utilizatorul tipic nu este expert auto, dar poate citi datele de bază ale unui anunț: marcă, model, an, kilometraj, motorizare și preț. Înainte de o vizionare fizică, acesta vrea să știe dacă oferta este rezonabilă în raport cu piața sau dacă merită tratată cu prudență.

Pentru definirea cerințelor a fost folosit un chestionar pilot realizat în Google Forms, adresat în principal unor studenți de la Facultatea de Automatică și Calculatoare care iau în calcul cumpărarea unei mașini second-hand. Eșantionul nu a fost tratat ca studiu statistic reprezentativ pentru întreaga piață, ci ca metodă exploratorie de produs. Scopul a fost identificarea întrebărilor reale pe care le are un cumpărător la începutul procesului de selecție.

Întrebările urmărite în formular au fost:

- ce site-uri sunt folosite cel mai des pentru căutarea unei mașini second-hand;
- cât timp petrec utilizatorii comparând manual anunțuri similare;
- ce informații îi interesează înainte de a suna vânzătorul;
- dacă un indicator de tip „preț corect / prea mic / prea mare” ar fi util;
- dacă utilizatorii ar dori să vadă și exemple de anunțuri similare, nu doar o estimare numerică;
- ce nivel de toleranță față de prețul estimat li se pare acceptabil.

Concluzia practică a acestei etape a fost că nevoia principală nu este o estimare perfectă, ci o filtrare preliminară. Utilizatorii vor să reducă rapid lista de anunțuri și să știe unde merită investit timp suplimentar. Din acest motiv, aplicația nu se limitează la afișarea unui preț estimat, ci oferă și verdict, diferență procentuală, predicții pe model și anunțuri similare.

2.2 Nevoi identificate

Din analiza utilizatorilor au rezultat patru nevoi importante. Prima este verificarea rapidă a corectitudinii prețului. Un cumpărător poate vedea zeci de anunțuri într-o singură seară, iar comparația manuală devine obositoare. A doua este detectarea ofertelor suspect de ieftine. În piața auto, un preț mult sub piață poate indica defecte, istoric neclar sau tentativă de fraudă.

A treia nevoie este explicabilitatea. Un simplu număr, de exemplu 12500 EUR, nu este suficient. Utilizatorul trebuie să vadă de ce rezultatul are sens: ce modele au estimat prețul, cât de apropiate sunt predicțiile și ce anunțuri similare există. A patra nevoie este păstrarea istoricului. Cumpărarea unei mașini presupune compararea mai multor opțiuni, iar utilizatorul are nevoie să revină la analizele anterioare.

2.3 Cerințe funcționale

- **Crearea și autentificarea contului.** Aplicația trebuie să permită crearea unui cont folosind email, username și parolă, urmată de autentificare. Contul este necesar pentru asocierea analizelor cu persoana care le-a efectuat și pentru păstrarea unui istoric separat.
- **Introducerea linkului de anunț.** Utilizatorul autentificat trebuie să poată introduce un link Autovit sau mobile.de. Folosirea linkului, în locul completării manuale a unui formular tehnic lung, reduce timpul necesar unei verificări și riscul introducerii greșite a datelor.
- **Extragerea automată a caracteristicilor.** Sistemul trebuie să extragă titlul, prețul, anul, kilometrajul, combustibilul, transmisia, puterea, capacitatea cilindrică și imaginea, atunci când sursa le publică. Aceste câmpuri sunt necesare atât pentru construirea vectorului de intrare al modelului, cât și pentru confirmarea vizuală că a fost analizat anunțul corect.
- **Estimarea prețului prin modele multiple.** Sistemul trebuie să calculeze estimări cu mai multe modele de regresie antrenate offline. Cerința evită dependența de o singură familie de algoritmi și permite observarea acordului sau dezacordului dintre modele.
- **Generarea verdictului.** Aplicația trebuie să transforme estimarea numerică într-un verdict ușor de interpretat: preț corect, preț suspect de mic sau preț prea mare. Verdictul răspunde direct întrebării formulate de utilizator și face rezultatul util chiar și pentru persoane fără experiență în machine learning.
- **Configurarea toleranței.** Utilizatorul trebuie să poată modifica toleranța procentuală folosită la verdict. O marjă fixă nu este potrivită pentru toate bugetele și segmentele auto, deoarece o diferență de 10% are altă semnificație pentru o mașină de 5000 EUR decât pentru una de 80000 EUR.
- **Alegerea metodei de combinare.** Interfața trebuie să permită alegerea între ponderarea invers proporțională cu MAE și varianta care ține cont suplimentar de acordul dintre modele. Această opțiune face vizibilă decizia de agregare și permite compararea celor

două strategii în aceeași aplicație.

- **Afișarea anunțurilor similare.** Rezultatul trebuie să includă exemple apropiate din baza de date colectată. Acestea oferă un reper concret de piață și îl ajută pe utilizator să verifice intuitiv dacă estimarea este plauzibilă.
- **Salvarea istoricului complet.** Fiecare predicție reușită trebuie salvată în istoricul contului împreună cu toleranța verdictului, metoda de agregare, estimările individuale, metricile și anunțurile similare. Cumpărarea unei mașini presupune de regulă compararea mai multor opțiuni, iar păstrarea raportului complet permite revenirea la contextul exact al deciziei fără recalculare.
- **Administrarea istoricului.** Utilizatorul trebuie să poată șterge o analiză individuală sau întregul istoric. Cerința oferă control asupra datelor contului și evită aglomerarea interfeței după multe verificări.
- **Explicații accesibile.** Interfața trebuie să ofere o pagină separată cu explicații simple pentru MAE, RMSE, R^2 , metoda de agregare, toleranță și similaritate. Butoanele de ajutor contextual trebuie să conducă utilizatorul către explicația relevantă fără a întrerupe fluxul principal.
- **Afișarea progresivă a rezultatului.** Raportul trebuie să prezinte inițial verdictul și estimarea finală ponderată. Predicțiile individuale și graficele de performanță trebuie să poată fi extinse la cerere, deoarece utilizatorul obișnuit are nevoie mai întâi de concluzie, iar detaliile tehnice sunt utile în etapa de verificare.

2.4 Cerințe nefuncționale

- **Utilizabilitate și acces de pe mai multe ecrane.** Interfața trebuie să fie în limba română, să păstreze un flux simplu și să se adapteze la desktop și mobil. Majoritatea utilizatorilor nu sunt specialiști tehnici, iar o aplicație de evaluare trebuie să poată fi folosită fără documentație prealabilă.
- **Feedback și timp de răspuns controlat.** Sistemul trebuie să ofere feedback vizual pe durata analizei și să încheie cererea cu rezultat sau eroare într-un interval limitat. Scraping-ul poate dura câteva secunde, dar animația nu trebuie să ruleze nelimitat atunci când platforma externă nu răspunde.
- **Reproductibilitatea datelor.** Datele brute trebuie păstrate în JSONL, iar seturile normalizate și metricile în formate auditabile precum CSV și JSON. Această cerință permite refacerea normalizării, verificarea deduplicării și repetarea antrenării fără colectarea inutilă a acelorași pagini.
- **Portabilitate între local și cloud.** Căile către modele, datasetul de similaritate și baza de date trebuie configurate prin variabile de mediu. Separarea configurației de cod permite rularea aceleiași aplicații pe calculatorul de dezvoltare și în containerul Docker publicat pe Render.

- **Securitatea autentificării.** Parolele nu trebuie salvate în clar, tokenurile de sesiune trebuie generate aleator, iar cookie-ul de autentificare trebuie să fie HTTP-only și secure în mediul HTTPS. Motivația este limitarea efectelor unei expuneri accidentale a bazei și reducerea accesului JavaScript la credențialele de sesiune.
- **Robustețe la erorile surselor externe.** Erorile de rețea, schimbările de structură HTML și blocările anti-bot trebuie tratate explicit. Aplicația trebuie să returneze un mesaj clar și să oprească starea de încărcare, deoarece disponibilitatea Autovit și mobile.de nu este controlată de sistem.
- **Trasabilitatea modelelor.** Bundle-ul salvat trebuie să conțină lista caracteristicilor, pipeline-urile și metricile asociate modelelor. Cerința este importantă pentru a reproduce o estimare și pentru a evita încărcarea unui model cu versiuni incompatibile ale bibliotecilor.
- **Limitarea ratei cererilor.** Endpoint-urile publice și operațiile costisitoare trebuie protejate împotriva unui număr excesiv de cereri. Autentificarea și înregistrarea sunt limitate după adresa IP, iar predicția după contul autentificat. Cerința reduce riscul încercărilor automate asupra parolelor, al creării abuzive de conturi și al consumării resurselor limitate ale serviciului Render.

2.5 Criterii de acceptare

Un flux este considerat funcțional dacă utilizatorul poate crea cont, se poate autentifica, poate analiza un link valid, poate vedea verdictul și poate redeschide din istoric raportul complet cu aceleași setări și comparații. Termenii tehnici trebuie să poată fi consultați fără părăsirea fluxului de analiză, iar detaliile modelelor trebuie să rămână opționale. La depășirea unei limite de cereri, aplicația trebuie să răspundă controlat cu statusul HTTP 429 și să indice timpul minim de așteptare. Pentru partea de modelare, acceptarea nu se bazează pe o predicție perfectă pentru fiecare anunț, ci pe obținerea unor metrici rezonabile pe setul de test și pe un comportament coerent în studiile de caz. Pentru partea de deployment, acceptarea minimă este ca ruta `/health` să confirme existența modelului, datasetului de similaritate și graficelor principale.

Capitolul 3

Studiu de piață și fundamentare

3.1 Evoluția pieței second-hand

Motivația proiectului este susținută de dimensiunea și dinamica pieței auto second-hand. În România, segmentul mașinilor second-hand din import a ajuns în 2024 la 344486 de unități înmatriculate, față de 309876 în 2023 și 325062 în 2022, conform unei analize publicate pe baza datelor DGCPI [18]. Aceeași sursă indică și faptul că reînmatriculările interne au depășit 710000 de unități în 2024, cel mai ridicat nivel din ultimii șase ani. Aceste valori arată că decizia de cumpărare a unei mașini second-hand nu este o situație izolată, ci o problemă frecventă pentru cumpărătorii români.

Evoluția lunară publicată de ACAROM confirmă intensitatea pieței auto românești: statisticile de înmatriculare separă autoturismele noi de vehiculele second-hand înmatriculate pentru prima dată în România și oferă un reper instituțional pentru dimensiunea fluxului de vehicule rulate [1]. În lucrare, aceste valori nu sunt folosite pentru antrenare, ci pentru justificarea contextului economic al aplicației.

La nivel european, literatura instituțională confirmă importanța pieței vehiculelor rulate. Un raport al Joint Research Centre arată că, în cele mai mari piețe auto din UE, mașinile noi reprezintă doar o parte din vânzările anuale totale, iar înțelegerea pieței second-hand este necesară pentru analiza flotei auto și a tranziției către vehicule mai noi sau mai eficiente [23]. Același raport evidențiază și dificultatea colectării unor date armonizate despre vehiculele rulate, ceea ce justifică folosirea unor surse publice și a unor pipeline-uri de normalizare în proiecte aplicative.

3.2 Autovit

Autovit oferă deja mecanisme de evaluare a prețului. Pagina oficială de evaluare menționează că instrumentul folosește detaliile esențiale ale vehiculului, compară mașina cu vehicule similare de pe platformă și calculează un interval de preț estimat [3]. Aceeași pagină indică peste 275000 de anunțuri utilizate pentru calcularea estimării și precizează că modelul este antrenat succesiv la fiecare două săptămâni, pentru a reflecta schimbările pieței [3].

Autovit publică și explicații mai generale despre evaluarea auto online. Într-un articol de blog, platforma menționează că sunt analizate informații despre vehicul și sunt comparate anunțuri similare publicate în ultimele 12 luni [2]. Acest detaliu este important pentru poziționarea proiectului CarValuator: indicatorul Autovit este foarte util în interiorul platformei, dar este legat de datele și regulile platformei respective.

În anunțuri, indicatorul Autovit are rol de semnal rapid pentru cumpărător, însă aplicația CarValuator urmărește o experiență diferită. Utilizatorul introduce un link, primește predicții de la mai multe modele, poate ajusta toleranța verdictului și vede exemple similare dintr-un dataset combinat. În plus, aplicația salvează istoricul analizelor pe cont, ceea ce transformă verificarea într-un proces comparativ.

3.3 mobile.de

mobile.de include la rândul său un indicator de preț. Documentația Seller API precizează că platforma poate returna un price rating pentru un vehicul și că oferta este evaluată prin analiza unei game largi de date despre vehicul și compararea cu oferte similare [14]. Răspunsul include o etichetă, de exemplu GOOD_PRICE, iar documentația explică faptul că etichetele descriu poziția prețului față de vehicule similare.

Documentația arată și limitele calculului. Price rating-ul este disponibil pentru categoria Car, nu este disponibil pentru mașini noi, vehicule avariate sau prea vechi, iar în cazul anumitor configurații poate lipsi din cauza datelor insuficiente [14]. Pentru dealeri, mobile.de oferă și un Insights API care compară listările cu vehicule similare și expune metrici de performanță ale anunțurilor, dar accesul este limitat la parteneri din programul de date al platformei [12].

Prin urmare, mobile.de are un sistem matur de indicatori interni, dar acesta nu este complet transparent pentru un utilizator extern și nu poate fi folosit liber de o aplicație academică fără acces partener. CarValuator nu încearcă să reproducă formula internă, ci construiește un model propriu pe date colectate public și documentează explicit limitările scraping-ului.

3.4 Poziționarea soluției propuse

Soluțiile existente sunt integrate în platformele comerciale și beneficiază de acces la date interne. Acesta este un avantaj major. În același timp, ele sunt orientate către propria platformă și nu oferă utilizatorului control asupra pragului de verdict sau asupra metodei de combinare a modelelor.

CarValuator este poziționat ca prototip academic independent. El nu concurează direct cu modelele comerciale, ci demonstrează un flux complet: colectare de date, normalizare, selecție statistică, antrenare ML, aplicație web, autentificare, istoric și deployment. Diferențele principale sunt sintetizate în [Tabelul 3.1](#).

Tabela 3.1: Comparație între abordări

Criteriu	Autovit	mobile.de	CarValuator
Sursă date	Date interne Autovit	Date interne mobile.de	Date publice colectate prin scraping
Domeniu	Platforma Autovit	Platforma mobile.de	Linkuri Autovit și mobile.de
Metodă publicată	Comparație cu anunțuri similare și model reantrenat periodic	Comparație cu oferte similare și price rating API	Modele ML antrenate local, cu metrici și grafice
Control utilizator	Limitat	Limitat	Toleranță și metodă de ponderare configurabile
Transparență în aplicație	Indicator scurt	Indicator scurt	Predicții individuale, medie ponderată, anunțuri similare

3.5 Riscuri și limite ale comparației

Comparația cu Autovit și mobile.de trebuie interpretată cu prudență. Platformele comerciale folosesc probabil date interne mai bogate, inclusiv semnale care nu sunt disponibile prin scraping public. De asemenea, indicatorii lor pot fi calculați numai pentru anunțuri care respectă anumite condiții de recentă, stare și completitudine. În schimb, CarValuator lucrează cu un snapshot academic și cu date parțial incomplete.

Acest lucru nu invalidează proiectul, ci îi definește scopul. Lucrarea nu afirmă că prototipul depășește sistemele comerciale, ci că un flux reproductibil de scraping și machine learning poate oferi utilizatorului o verificare suplimentară, explicabilă și independentă de o singură platformă.

3.6 Fundamentare teoretică și lucrări conexe

Această secțiune prezintă conceptele pe care se bazează soluția propusă: formularea estimării ca problemă de regresie, familiile de modele analizate, transformarea țintei, selecția caracteristicilor și metricele de evaluare. Sunt discutate și particularitățile colectării automate a datelor, împreună cu tehnologiile folosite în implementare.

3.6.1 Evaluarea prețurilor ca problemă de regresie

Estimarea prețului unui autoturism second-hand poate fi formulată ca o problemă de regresie supravegheată. Pentru fiecare anunț există un vector de caracteristici x , format din informații precum anul, kilometrajul, puterea, combustibilul și marca, și o valoare țintă y , reprezentată de prețul publicat. Scopul modelului este să aproximeze o funcție $f(x)$ care produce o estimare apropiată de y .

Această formulare este inspirată de teoria prețurilor hedonice, unde prețul unui bun diferențiat este explicat prin caracteristicile sale [19]. Pentru mașini, caracteristicile tehnice și comerciale au efecte intuitive: o mașină mai nouă este, în general, mai scumpă; un kilometraj mai mare reduce valoarea; o putere mai mare poate crește prețul, dar numai în combinație cu marca, segmentul și vechimea.

Regresia nu poate captura toate elementele care influențează valoarea reală. Starea caroseriei, istoricul accidentelor, calitatea reparațiilor, numărul de proprietari, dotările exacte sau încrederea în vânzător sunt informații greu de extras automat. De aceea, predicția trebuie interpretată ca estimare statistică, nu ca expertiză tehnică.

3.6.2 Modele folosite

Proiectul compară mai multe familii de modele. Alegerea nu urmărește doar obținerea celui mai bun scor, ci și observarea comportamentului modelelor pe date tabulare reale, incomplete și neuniforme.

SVR, sau Support Vector Regression, este o extensie a mașinilor cu vectori suport pentru probleme de regresie. Modelul caută o funcție cât mai plată care tolerează erori mici în interiorul unui tub definit de parametrul ϵ , penalizând abaterile mai mari [22]. În proiect a fost folosit un kernel RBF, util atunci când relația dintre caracteristici și preț este neliniară.

Ridge Regression este o regresie liniară regularizată. Ea oferă un reper simplu și stabil, mai ales când datele conțin multe variabile categorice transformate prin one-hot encoding. Chiar dacă relația reală este neliniară, Ridge poate funcționa surprinzător de bine atunci când există suficiente caracteristici explicative.

KNN Regression estimează prețul prin vecinii cei mai apropiați ai anunțului analizat. Pentru date auto, ideea este intuitivă: o mașină ar trebui comparată cu mașini asemănătoare. Totuși, KNN este sensibil la scalare, la variabile categorice rare și la densitatea datelor în vecinătatea unui exemplu.

Random Forest construiește un ansamblu de arbori de decizie antrenați pe eșantioane diferite

și pe subseturi ale caracteristicilor [4]. Acest model este robust la neliniarități și interacțiuni, dar poate produce modele mari și poate extrapola slab pentru exemple mult diferite de setul de antrenare.

Extra Trees introduce o randomizare mai puternică în alegerea pragurilor de împărțire, reducând varianța și crescând diversitatea arborilor [8]. Gradient Boosting construiește arbori secvențial, fiecare încercând să corecteze erorile runde precedente [7]. În final, Voting Regressor combină estimările mai multor modele, încercând să reducă dependența de o singură ipoteză.

3.6.3 Transformarea logaritmică a prețului

Prețurile autoturismelor sunt distribuite asimetric. Majoritatea anunțurilor se află în zona medie a pieței, în timp ce un număr mic de vehicule premium sau foarte noi au prețuri mult mai mari. Dacă modelul este antrenat direct pe preț, exemplele scumpe pot domina funcția de eroare.

Această asimetrie poate fi observată în [Figura 5.2](#), unde cea mai mare parte a anunțurilor este concentrată în intervalele inferioare și medii de preț, iar valorile ridicate formează o coadă lungă a distribuției.

Pentru a reduce această problemă, pipeline-ul folosește ținta:

$$y_{log} = \log(1 + y)$$

După predicție, valoarea este readusă în EUR folosind:

$$\hat{y} = \exp(\hat{y}_{log}) - 1$$

Această transformare are două avantaje. În primul rând, comprimă intervalul valorilor și face erorile relative mai echilibrate. În al doilea rând, reduce tendința modelelor de a supraestima mașinile ieftine sau de a subestima exagerat mașinile foarte scumpe. În evaluarea proiectului, antrenarea pe log-preț a produs predicții mai stabile decât primele încercări pe preț brut.

3.6.4 Selecția caracteristicilor

Seturile de date obținute prin scraping conțin multe câmpuri, însă nu toate sunt utile pentru predicție. Unele câmpuri lipsesc frecvent, altele sunt prea specifice, iar altele au legătură slabă cu prețul. Pentru a evita introducerea de zgomot, pipeline-ul aplică o selecție de caracteristici înainte de modelare.

Pentru variabile numerice se folosește corelația Pearson, o măsură clasică a asocierii liniare dintre două variabile [15]. Pentru variabile categorice, ținta numerică este discretizată în intervale, apoi se aplică un test de tip chi-pătrat. Mărimea efectului este aproximată prin Cramer V, o măsură potrivită pentru tabele de contingență [5].

Selecția nu este tratată ca adevăr absolut. De exemplu, transmisia și tipul de caroserie sunt

caracteristici importante în lumea reală, dar în setul combinat disponibil ele apar foarte rar pentru Autovit. Dacă o coloană are puține valori completate, testul statistic poate decide eliminarea ei nu pentru că este irelevantă conceptual, ci pentru că datele curente nu permit estimarea efectului.

3.6.5 Metrici de evaluare

Pentru evaluare sunt folosite trei metrice. RMSE penalizează mai mult erorile mari, fiind util pentru a observa cazurile în care modelul ratează puternic. MAE măsoară eroarea absolută medie și este mai ușor de interpretat în EUR. R^2 exprimă proporția variației țintei explicată de model, comparativ cu o predicție constantă bazată pe medie [20].

În contextul aplicației, MAE este deosebit de importantă deoarece utilizatorul gândește în diferențe concrete de bani. O eroare de 3000 EUR poate fi acceptabilă pentru o mașină de 80000 EUR, dar foarte mare pentru una de 7000 EUR. Din acest motiv, verdictul final folosește și un prag procentual configurabil de utilizator.

3.6.6 Web scraping și considerații etice

Web scraping-ul permite colectarea automată a datelor publicate pe pagini web, însă ridică probleme tehnice, legale și etice. Krotov și Silva subliniază că cercetătorii trebuie să analizeze scopul colectării, tipul datelor, impactul asupra site-ului și riscurile de confidențialitate [10].

În proiectul CarValuator, datele au fost folosite în scop academic, iar colectarea a fost limitată la câmpuri publice din anunțuri. Informațiile de contact nu sunt necesare pentru modelare și nu sunt folosite pentru predicție. Scraping-ul a fost rulat cu întârzieri între cereri, iar pipeline-ul păstrează datele brute tocmai pentru a evita repetarea inutilă a solicitărilor către site-uri.

Totuși, există o diferență importantă între un prototip local și un serviciu public. Dacă aplicația este publicată în cloud, fiecare predicție live poate genera o cerere server-side către site-ul anunțului. Acest lucru poate duce la limitări sau blocări, mai ales pentru mobile.de, unde mecanismele anti-automatizare sunt mai agresive.

3.6.7 Context tehnologic și alegerea platformei de deployment

Implementarea folosește Python și scikit-learn pentru pipeline-ul ML. scikit-learn este o bibliotecă standard pentru învățare automată în Python, cu implementări mature pentru regresie, preprocesare și evaluare [16]. Pentru aplicația web, backend-ul este construit cu FastAPI, un framework modern bazat pe tipuri Python și OpenAPI [6]. Pentru cazurile în care este necesară interacțiunea cu pagini dinamice, proiectul folosește Playwright, care oferă automatizare de browser pentru Python [11].

Render a fost ales pentru publicarea prototipului deoarece acceptă servicii definite prin Docker și conectate la un repository Git. Platforma permite configurarea separată a variabilelor de mediu și redeploy automat la actualizarea codului [17]. Pentru un proiect academic, aceste facilități reduc administrarea manuală a unui server și permit oferirea rapidă a unei adrese publice pentru demonstrații și teste de utilizabilitate. Folosirea containerului Docker păstrează

aceleași versiuni Python, scikit-learn și Playwright între mediul local și cel din cloud, reducând riscul ca modelele serializate sau browserul automatizat să funcționeze diferit după deployment. Alegerea este una pragmatică pentru demonstrarea aplicației, nu o afirmație că Render ar fi soluția optimă pentru un sistem de producție la scară mare.

Capitolul 4

Proiectarea aplicației

4.1 Obiective de proiectare

Proiectarea aplicației CarValuator urmărește separarea clară între colectarea datelor, pregătirea lor, antrenarea modelelor și folosirea modelului într-o interfață web. Această separare este necesară deoarece cele patru activități au ritmuri diferite. Scraping-ul și antrenarea sunt operații offline, costisitoare și repetate controlat, în timp ce analiza unui link trebuie să fie o operație online, suficient de rapidă pentru utilizator.

Sistemul a fost proiectat ca un monolit modular. Backend-ul FastAPI servește API-ul și fișierele statice HTML, CSS și JavaScript, iar modulele Python rămân independente la nivel logic. Alegerea reduce complexitatea deployment-ului pentru un prototip academic, fără să amestece responsabilitățile interne. Datasetul, bundle-ul de modele și baza SQLite sunt artefacte separate, configurabile prin variabile de mediu.

Alte două obiective au fost trasabilitatea și toleranța la defecte. O predicție trebuie să poată fi explicată prin datele normalizate și estimările modelelor, iar o eroare produsă de un site extern nu trebuie să blocheze sesiunea sau interfața.

4.2 Arhitectura modulară

Figura 4.1 prezintă schema bloc a sistemului. Arhitectura conține următoarele module:

- **frontend web**, responsabil de autentificare, introducerea linkului, configurarea toleranței și afișarea rezultatului;
- **API FastAPI**, care validează cererile, verifică sesiunea și orchestrează fluxul de analiză;
- **scrapere specifice sursei**, care transformă un URL Autovit sau mobile.de într-o înregistrare brută;
- **normalizare și feature engineering**, care convertește valorile eterogene într-o schemă comună;

- **pipeline de antrenare**, care selectează caracteristici, compară algoritmi și salvează metrice;
- **modul de inferență**, care încarcă bundle-ul și calculează estimările individuale și agregate;
- **motor de similaritate**, care caută anunțuri apropiate în CSV-ul configurat;
- **persistență SQLite**, folosită pentru utilizatori, sesiuni și istoricul analizelor;
- **artefacte de deployment**, alcătuite din modelul comprimat, datasetul de similaritate și grafice.

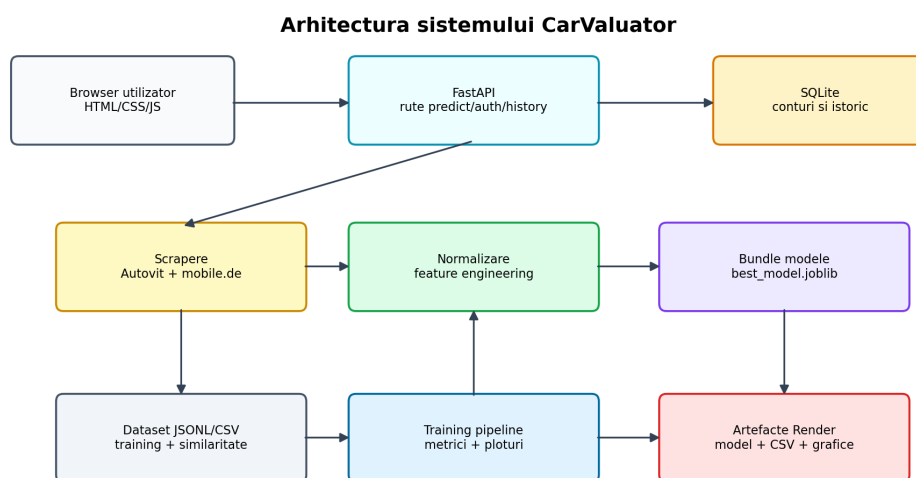


Figura 4.1: Arhitectura modulară a sistemului CarValuator

Legăturile dintre module au fost păstrate unidirecționale pe cât posibil. Interfața nu accesează direct modelele sau baza de date, iar scraperul nu decide verdictul. API-ul este punctul de coordonare, însă logica de modelare rămâne în modulul de inferență, ceea ce permite testarea sa și din linia de comandă.

4.3 Cazuri de utilizare

- **Crearea contului.** Vizitatorul introduce email, username și parolă. Sistemul validează datele, calculează hash-ul parolei, creează utilizatorul și deschide o sesiune.
- **Autentificarea.** Utilizatorul furnizează emailul sau username-ul și parola. Dacă verificarea reușește, backend-ul creează un token de sesiune și îl transmite prin cookie HTTP-only.
- **Analiza unui anunț.** Utilizatorul introduce linkul, metoda de agregare și toleranța. Sistemul identifică platforma, extrage sau caută local anunțul, normalizează caracteristicile, rulează modelele și afișează verdictul.

- **Verificarea explicațiilor.** După predicție, utilizatorul vede mai întâi estimarea finală și verdictul. La cerere poate extinde predicțiile algoritmilor și graficele, iar butoanele de ajutor explică pe scurt metricile și regulile de agregare. Pagina separată de explicații oferă o referință accesibilă fără a aglomera raportul principal.
- **Administrarea istoricului.** Utilizatorul deschide pagina dedicată istoricului, vizualizează analizele asociate contului și redeschide raportul complet. Raportul păstrează setările verdictului, metoda de agregare, estimările modelelor și sugestiile similare. Utilizatorul poate deschide anunțul original și poate șterge un element sau întregul istoric.
- **Diagnosticarea deployment-ului.** Administratorul accesează ruta `/health` pentru a verifica existența modelului, a datasetului și versiunile principalelor biblioteci.

4.4 Fluxul de execuție al unei predicții

Figura 4.2 este o diagramă UML de secvență pentru cazul principal. Frontend-ul trimite o singură cerere `POST /predict`. API-ul verifică sesiunea în SQLite, apelează scraperul potrivit, normalizează datele și transmite vectorul către modelele ML. După agregare, rezultatul este salvat în istoric și returnat ca JSON.

Diagrama de secvență pentru analiza unui anunț

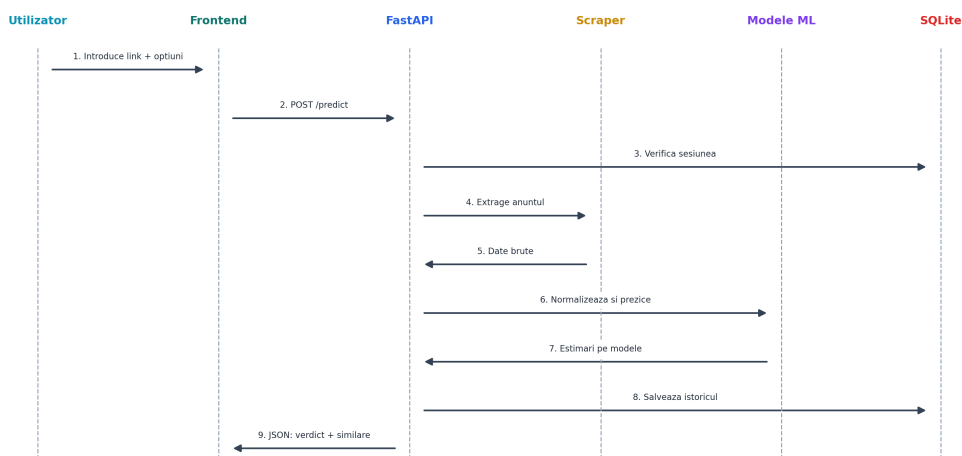


Figura 4.2: Diagramă UML de secvență pentru analiza unui anunț

Pentru `mobile.de` a fost proiectat un flux cu fallback. Mai întâi este căutat identificatorul anunțului în datasetul local, apoi se încearcă pagina de detaliu, iar căutarea live este ultima opțiune. Această ordine reduce timpul de răspuns și dependența de protecțiile anti-bot. În cloud, accesul live poate fi refuzat de platformă, astfel că anunțurile existente în snapshot-ul livrat sunt cazul cel mai stabil.

4.5 Proiectarea API-ului

FastAPI a fost ales deoarece permite definirea rutelor și validarea request-urilor prin modele Pydantic [6]. Contractul principal primește sursa, URL-ul, toleranța, metoda de agregare și numărul maxim de anunțuri similare. Răspunsul conține prețul publicat, estimarea finală, verdictul, datele normalizate, predicțiile individuale și lista de comparații.

Tabela 4.1: Rutele proiectate pentru aplicația web

Rută	Metodă	Responsabilitate
/health	GET	Verifică artefactele și versiunile runtime
/auth/register	POST	Creează un utilizator și o sesiune
/auth/login	POST	Verifică parola și creează sesiunea
/auth/logout	POST	Invalidează sesiunea curentă
/predict	POST	Execută analiza completă și salvează rezultatul
/history	GET	Returnează istoricul utilizatorului curent
/history/{id}	GET	Returnează raportul complet deținut de utilizator
/history/{id}	DELETE	Șterge o analiză deținută de utilizator
/history	DELETE	Șterge toate analizele contului

Codurile de stare separă erorile de utilizare de cele operaționale. Un URL invalid produce un răspuns 400, lipsa autentificării produce 401, iar eșecul unei surse externe este raportat separat de lipsa modelului. Această proiectare simplifică afișarea unui mesaj adecvat în frontend.

4.6 Proiectarea navigării

Interfața este împărțită în trei pagini principale servite de același backend: evaluarea anunțului, istoricul contului și explicațiile. Separarea reduce încărcarea cognitivă a paginii de evaluare. Bara de navigare rămâne disponibilă pe toate paginile, iar starea autentificării este comună deoarece sesiunea este transmisă prin cookie.

[Figura 4.3](#) organizează funcționalitățile de navigare după două criterii: numărul estimat de utilizatori care le folosesc și frecvența utilizării de către o singură persoană. Verificarea unui anunț reprezintă acțiunea centrală și este folosită frecvent de toți utilizatorii. Autentificarea este utilizată ocazional, de regulă o singură dată într-o sesiune, iar crearea contului este o acțiune rară, dar necesară fiecărui utilizator. Consultarea istoricului este folosită ocazional de mulți utilizatori, în timp ce pagina de explicații este accesată rar și de un grup mai restrâns. Matricea a fost folosită pentru a decide ce legături trebuie să rămână permanent în bara principală și ce funcții pot fi accesate contextual.

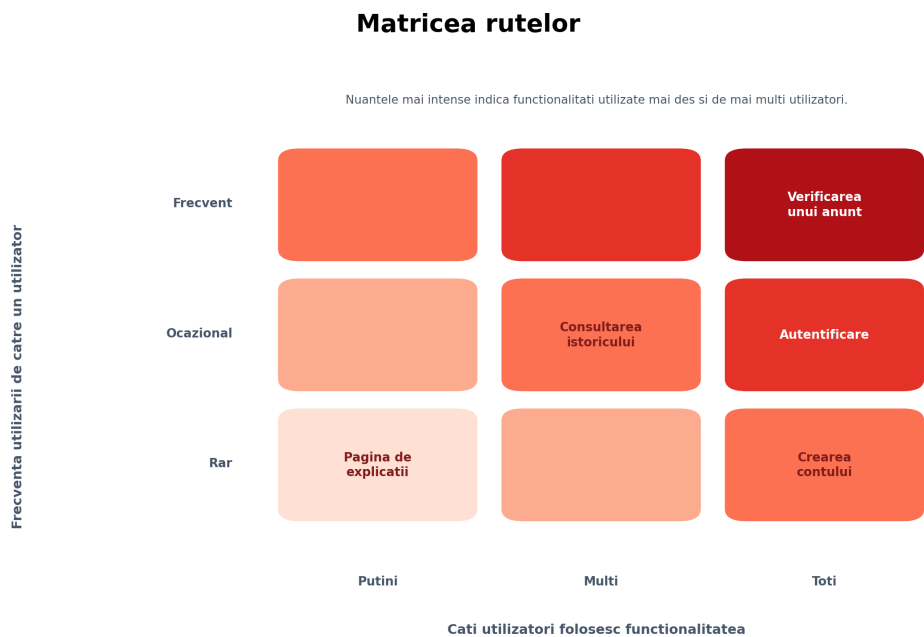


Figura 4.3: Matricea funcționalităților în funcție de răspândirea și frecvența utilizării

4.7 Proiectarea bazei de date

Baza SQLite are trei tabele și două relații *one-to-many*, prezentate în Figura 4.4. SQLite a fost suficient pentru prototip deoarece aplicația are un singur proces și un volum redus de scrieri. Structura poate fi migrată ulterior către PostgreSQL fără schimbarea conceptelor principale.

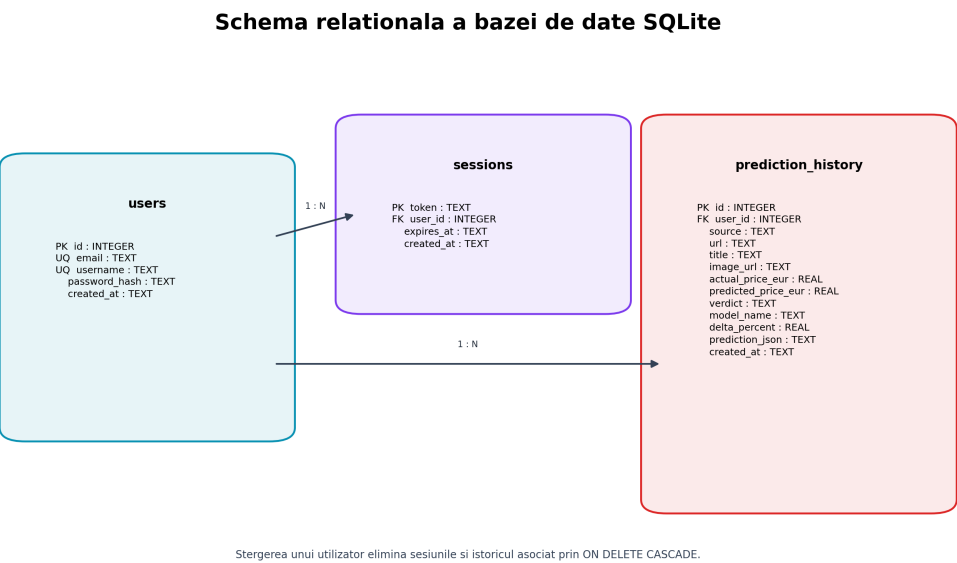


Figura 4.4: Schema relațională pentru utilizatori, sesiuni și istoricul predicțiilor

Tabela `users` păstrează identificatorul, emailul și username-ul unice, hash-ul parolei și momentul creării. Separarea parolei sub formă de hash face imposibilă recuperarea directă a parolei originale din bază.

Tabela `sessions` folosește tokenul aleator drept cheie primară și conține cheia externă `user_id`, expirarea și momentul creării. Mai multe sesiuni pot aparține aceluiași cont, de exemplu pentru browsere diferite.

Tabela `prediction_history` păstrează câmpurile necesare afișării rapide, precum titlul, prețurile, verdictul și imaginea. Coloana `prediction_json` păstrează răspunsul complet, inclusiv toleranța, metoda de agregare, datele normalizate, metricile modelelor și sugestiile similare. Astfel, analiza poate fi redeschisă identic fără o nouă cerere către site-ul extern. Ștergerea este condiționată de perechea `id` și `user_id`, prevenind eliminarea istoricului altui cont.

4.8 Proiectarea pentru securitate și defecte

Parolele sunt protejate prin PBKDF2 cu salt și număr mare de iterații, iar comparația hash-urilor este realizată în timp constant. Tokenurile de sesiune sunt generate cu o sursă criptografică și nu conțin date despre utilizator. Cookie-ul este HTTP-only, folosește politica SameSite și este marcat secure în deployment-ul HTTPS.

În fața rutelor API este proiectat un modul de limitare a ratei cu fereastră glisantă. Limita generală folosește adresa IP, autentificarea și înregistrarea folosesc tot IP-ul, iar predicțiile folosesc identificatorul contului. Separarea cheilor permite mai multor colegi să testeze aplicația de pe dispozitive diferite fără ca un singur cont să poată monopoliza inferența. La depășire, API-ul returnează statusul 429 și antetul `Retry-After`. Valorile sunt configurabile prin variabile de mediu, deoarece limitele potrivite pentru un demo diferă de cele ale unui serviciu cu resurse dedicate.

Sursele externe sunt considerate componente nesigure. Scraperul poate primi răspunsuri incomplete, erori 403 sau pagini cu structură schimbată. Din acest motiv, câmpurile sunt opționale până la validare, iar inferența este pornită numai dacă există informațiile minime. Frontend-ul afișează o stare de încărcare, dar cererea trebuie să aibă un timeout și să închidă animația și în cazul unei erori.

4.9 Decizii de deployment

Deployment-ul este proiectat pentru Render printr-un container Docker. Containerul instalează versiunile bibliotecilor folosite la serializarea modelului și browserul Chromium necesar fallback-ului Playwright [11]. Configurația citește portul furnizat de Render și căile artefactelor din variabile de mediu [17].

Modelul complet local conține șapte pipeline-uri și ocupă aproximativ 606 MB. Pentru serviciul gratuit a fost proiectat un bundle de aproximativ 1 MB care păstrează SVR, Ridge, KNN și Gradient Boosting. Datasetul de antrenare rămâne separat de snapshot-ul operațional folosit

pentru sugestii. Snapshot-ul curent conține 2942 de anunțuri Autovit deduplicate, colectate la 12 iunie 2026, și este copiat identic în artefactele locale și Render. Reducerea bundle-ului scade memoria și timpul de pornire, însă înseamnă că interfața publică nu rulează Random Forest, Extra Trees și Voting Regressor.

Baza SQLite este plasată în /tmp pentru demo, ceea ce înseamnă că utilizatorii și istoricul pot fi pierduți la restart. Pentru producție, proiectarea prevede înlocuirea cu o bază persistentă sau montarea unui disc. Limitarea este acceptabilă doar pentru demonstrația academică.

Contoarele sunt păstrate în memoria procesului. Soluția este suficientă pentru instanța unică de demonstrație și evită o dependență suplimentară. Într-o arhitectură cu mai multe instanțe, proiectarea trebuie extinsă cu un backend comun, de exemplu Redis, pentru ca toate procesele să observe aceleași contoare.

Capitolul 5

Implementarea aplicației

Implementarea urmează modulele definite în Capitolul 4. Limbajul principal este Python, folosit pentru scraping, procesarea datelor, antrenare și backend. Interfața este realizată cu HTML, CSS și JavaScript fără framework frontend, pentru a păstra deployment-ul compact. Codul este organizat în pachetul `carvaluator_scraper`, iar comenzile de reproducere sunt prezentate în Anexa A.

5.1 Colectarea și pregătirea datelor

Implementarea componentei de date urmărește un flux complet, de la colectarea anunțurilor până la obținerea unui set coerent pentru antrenare. Subsecțiunile următoare descriu sursele, particularitățile scraperelor, schema comună, normalizarea, deduplicarea și verificările efectuate înaintea modelării.

5.1.1 Sursele de date

Datele proiectului provin din anunțuri publice de pe Autovit și mobile.de. Alegerea acestor două surse are o justificare practică. Autovit este relevant pentru piața românească, iar mobile.de este una dintre platformele europene importante pentru autoturisme second-hand, inclusiv pentru cumpărătorii români care caută mașini din Germania.

Setul combinat folosit pentru evaluarea finală conține 9633 de rânduri păstrate după normalizare și deduplicare. Din acestea, 7057 provin din Autovit, iar 2576 din mobile.de. Creșterea volumului mobile.de a fost obținută printr-o metodă experimentală bazată pe pagini SEO convertite în Markdown, descrisă în secțiunea dedicată acestei surse.

Figura 5.1 prezintă distribuția surselor. Setul rămâne dominat de Autovit, dar ponderea mobile.de este suficient de mare pentru a contribui vizibil la distribuțiile de preț și la antrenarea finală.

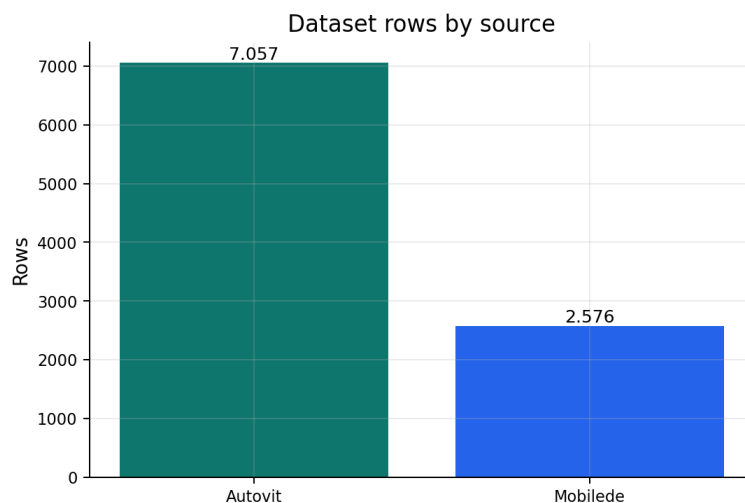


Figura 5.1: Componenta setului de date după sursă

Tabela 5.1: Rezumatul setului combinat înainte de curățarea finală pentru antrenare

Indicator	Valoare
Rânduri încărcate	10137
Rânduri păstrate după deduplicare	9633
Duplicate exacte eliminate	291
Duplicate fuzzy eliminate	213
Rânduri fără preț	9
Mărci distincte	82
Modele distincte	693
Mediană preț brut	16640 EUR
Mediană kilometraj brut	112927 km

5.1.2 Scraping pentru Autovit

Pentru Autovit, scraperul a fost construit pornind de la paginile de căutare. Fiecare pagină conține mai multe anunțuri, iar structura HTML include informații suficiente pentru extragerea linkului, titlului, prețului, anului, kilometrajului și câtorva câmpuri tehnice. În unele cazuri, datele sunt disponibile și în structuri JSON incluse în pagină, ceea ce permite o extragere mai stabilă decât parsarea textului vizibil.

Fluxul de scraping salvează fiecare anunț ca obiect JSONL. Alegerea JSONL este intenționată: fiecare linie este un document independent, iar fișierul poate fi procesat incremental. Dacă scraping-ul se oprește la pagina 80 din 150, datele deja salvate rămân valide. În plus, JSONL păstrează și câmpul `raw`, unde pot fi arhivate detalii specifice site-ului. Această decizie a fost utilă atunci când unele câmpuri au trebuit reinterpretate fără a reporni colectarea.

În practică, Autovit a ridicat mai multe dificultăți. Prima dificultate a fost instabilitatea structurii paginilor. Numele claselor HTML și poziția câmpurilor se pot modifica fără avertisment, iar scraperul trebuie să fie tolerant la câmpuri lipsă. A doua dificultate a fost deduplicarea.

Anunțurile promovate sau republicate pot apărea pe mai multe pagini, iar identificatorul de listing nu este întotdeauna suficient pentru eliminarea tuturor cazurilor similare. A treia dificultate a fost legată de imagini. Unele imagini sunt încărcate lazy, iar alte imagini pot reprezenta elemente de dealer, nu mașina propriu-zisă.

5.1.3 Scraping pentru mobile.de

Integrarea mobile.de a fost mai dificilă decât integrarea Autovit. Site-ul are localizare pe mai multe limbi, pagini dinamice, protecții anti-bot și comportament diferit între paginile de căutare și paginile de detaliu. În multe încercări, cererile automate au returnat erori 401, 403 sau 429. Aceste coduri indică lipsă de autorizare, interdicție de acces sau limitare de rată.

Pentru a crește șansele de colectare, scraperul mobile.de încearcă să folosească endpoint-uri JSON cu headere de browser. În plus, există un fallback cu Playwright, util atunci când pagina trebuie randată într-un browser real. Totuși, fallback-ul nu garantează succesul, deoarece protecțiile pot detecta mediile automate sau pot afișa pagini de consimțământ și acces restricționat.

O problemă importantă a apărut și la imaginile anunțurilor mobile.de. Inițial, aplicația putea afișa imaginea greșită, de exemplu bannerul dealerului sau fotografia de showroom, nu fotografia principală a mașinii. Soluția a fost prioritizarea imaginilor din galeria anunțului și filtrarea surselor care par logo-uri sau imagini de dealer. Această problemă arată că scraping-ul nu înseamnă doar extragerea unui câmp, ci și interpretarea corectă a contextului în care apare acel câmp.

Din cauza acestor dificultăți, primele date mobile.de colectate au fost puține. Integrarea a rămas însă valoroasă pentru aplicație deoarece permite analiza unor linkuri de detaliu și expune limitele reale ale unui sistem care depinde de site-uri externe.

Într-o etapă ulterioară, am testat și o strategie alternativă pentru mărirea volumului de date mobile.de. API-ul oficial de căutare mobile.de există, dar accesul la anunțuri necesită autentificare Basic și un cont cu drepturi pentru serviciul respectiv [13]. Fără aceste credențiale, endpoint-ul de căutare a răspuns cu 401, în timp ce endpoint-urile publice de referință au returnat doar liste de mărci și categorii, nu anunțuri concrete.

Metoda care a funcționat practic a fost citirea paginilor SEO publice de forma <https://www.mobile.de/en/car/marca/model> printr-un serviciu de conversie HTML în Markdown, Jina Reader [9]. În loc să controleze un browser, scriptul descarcă reprezentarea Markdown a paginii, apoi parsează cardurile de anunțuri. Din fiecare card se pot extrage titlul, prețul, anul primei înmatriculări, kilometrajul, puterea, combustibilul, orașul și prima imagine. Pentru cilindree, cardurile nu oferă mereu un câmp separat, astfel că parserul folosește o regulă conservatoare din titlu, de exemplu 2.0 TDI, 1.6 HDI sau 1.5 TSI. Regula evită codurile comerciale precum 35 TFSI și expresiile de consum precum 5,5 l/100km. Pentru că aceste carduri nu expun întotdeauna ID-ul real al anunțului, scriptul generează un identificator sintetic pe baza unei amprente formate din titlu, marcă, model, an, kilometraj, putere, preț și oraș.

Prin această metodă au fost citite 118 pagini de marcă sau model și au fost obținute 2735

rânduri brute mobile.de, după filtrarea blocurilor care nu reprezentau anunțuri. După exportul prin pipeline-ul de normalizare și deduplicare fuzzy au rămas 2534 rânduri curate din metoda Markdown. Combinarea cu lotul mobile.de obținut anterior a produs 2576 rânduri curate. În acest set mobile.de combinat, puterea este disponibilă pentru 2533 rânduri, iar capacitatea cilindrică pentru 647 rânduri. Această metodă nu înlocuiește API-ul oficial și nu este ideală pentru producție, deoarece depinde de un serviciu intermediar și nu oferă toate câmpurile tehnice, precum transmisia sau caroseria. Totuși, pentru un prototip academic, ea oferă un snapshot suplimentar util și reproductibil, salvat local în JSONL și CSV.

Pentru ca aceste rânduri să fie folosite la antrenare, filtrarea finală a fost ajustată. Valorile numerice imposibile sunt eliminate în continuare, însă câmpurile tehnice lipsă, precum capacitatea cilindrică, nu mai duc automat la eliminarea rândului. Ele sunt imputate ulterior de pipeline-ul de preprocesare. Această schimbare este importantă deoarece paginile Markdown oferă preț, an, kilometraj și putere, dar nu întotdeauna detalii complete despre motor.

5.1.4 Schema datelor brute

Fiecare rând brut conține câmpuri comune și câmpuri specifice sursei. Câmpurile comune sunt folosite ulterior pentru normalizare și modelare. [Tabelul 5.2](#) prezintă câteva exemple.

Tabela 5.2: Câmpuri extrase din anunțuri

Câmp	Rol
source	Platforma de proveniență, de exemplu autovit sau mobilede
listing_id	Identificatorul anunțului, atunci când poate fi extras
url	Linkul original al anunțului
title	Titlul afișat în anunț
make,model,version	Marca, modelul și versiunea vehiculului
year,mileage_km	Anul și kilometrajul
price_eur	Prețul convertit sau standardizat în EUR
fuel_type,transmission	Câmpuri tehnice categorice
power_hp,engine_capacity_cm3	Putere și capacitate cilindrică
seller_type,location_city	Context comercial și geografic

5.1.5 Normalizare

Datele brute sunt utile pentru audit, dar nu pot fi folosite direct în modele. Prețurile pot fi scrise cu separatori diferiți, kilometrajul poate conține text, combustibilul poate avea variante lingvistice, iar unele câmpuri pot lipsi. Normalizarea transformă aceste reprezentări într-un format comun.

Prețul este convertit în `price_eur`. În datele curente, majoritatea anunțurilor sunt deja în EUR, dar câmpul păstrează explicit moneda originală pentru extensii viitoare. Kilometrajul este extras ca număr întreg în kilometri. Puterea este standardizată în cai putere, iar capacitatea cilindrică în centimetri cubi. Pentru combustibil, valorile sunt mapate în categorii precum `diesel`, `petrol`, `hybrid`, `plug_in_hybrid` și `electric`.

Normalizarea construiește și câmpuri auxiliare. `dedupe_exact_key` identifică duplicatele cu același ID de sursă. `dedupe_fuzzy_key` combină marca, modelul, anul, kilometrajul, prețul și vânzătorul într-o amprentă conservatoare. `completeness_score` numără câte câmpuri relevante sunt prezente, fiind util pentru filtrarea rândurilor slabe.

5.1.6 Deduplicare

Deduplicarea are două niveluri. Primul nivel elimină duplicate exacte, folosind cheia de sursă și identificatorul anunțului. Al doilea nivel elimină duplicate probabile. Acesta este necesar deoarece același vehicul poate apărea cu identificatori diferiți sau poate fi republicat.

Cheia fuzzy este construită astfel încât să fie conservatoare. Dacă două anunțuri au aceeași marcă, același model, același an, kilometraj foarte asemănător, preț foarte asemănător și același vânzător sau oraș, este probabil să fie același vehicul. În setul combinat extins, acest pas a eliminat 213 rânduri suplimentare după eliminarea duplicatelor exacte. Numărul a crescut față de experimentul inițial deoarece paginile SEO mobile.de pot afișa același anunț în mai multe pagini de marcă sau model.

5.1.7 Analiza exploratorie

Analiza exploratorie arată că datele sunt neuniforme. Distribuția prețurilor este asimetrică, cu multe anunțuri în zona 5000-25000 EUR și câteva vehicule premium cu valori mult mai mari. [Figura 5.2](#) ilustrează această distribuție.

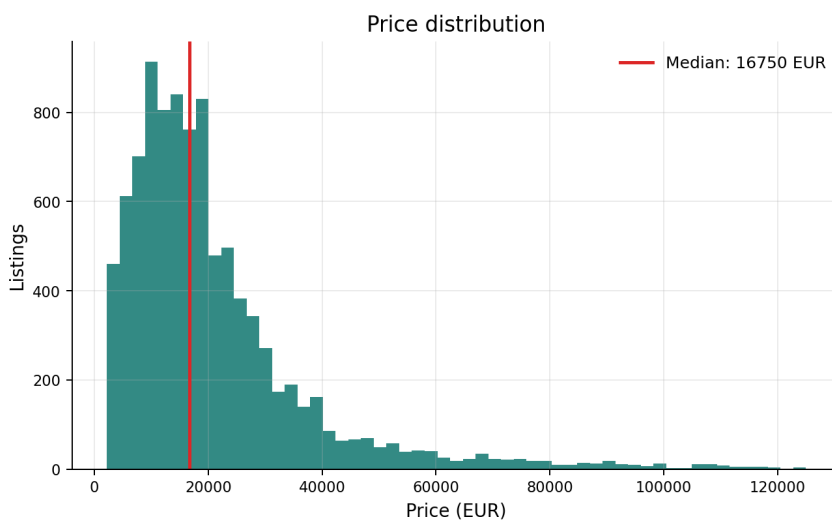


Figura 5.2: Distribuția prețurilor în setul de date

Kilometrajul și prețul au o relație negativă, dar relația nu este perfectă. Mașinile noi cu kilometraj mic sunt mai scumpe, însă marca și segmentul pot schimba semnificativ poziția unui anunț. O mașină premium cu kilometraj mare poate rămâne mai scumpă decât o mașină de oraș cu kilometraj mic. [Figura 5.3](#) arată această relație.

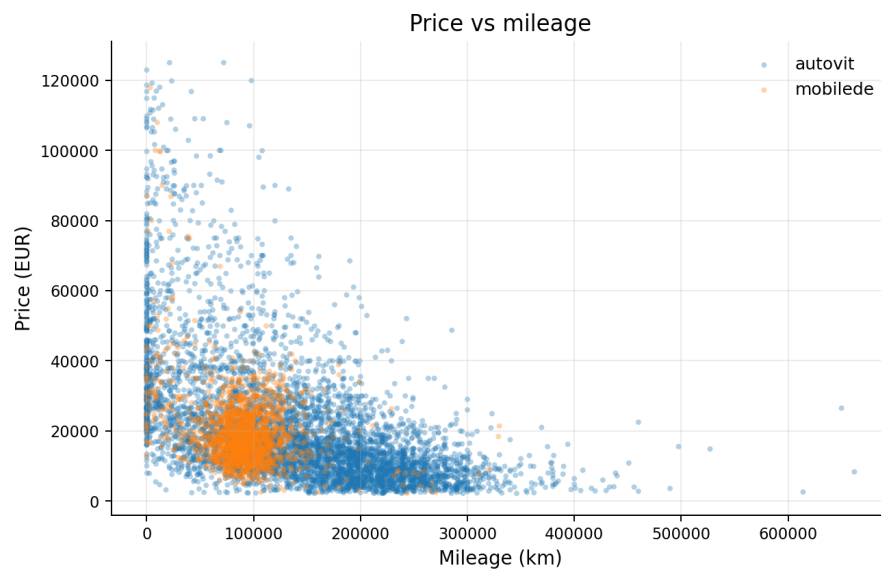


Figura 5.3: Prețul în funcție de kilometraj

Figura 5.4 arată evoluția mediane prețului în funcție de anul mașinii. Tendința generală este crescătoare pentru mașinile mai noi, dar există variații cauzate de compoziția pe mărci și modele.

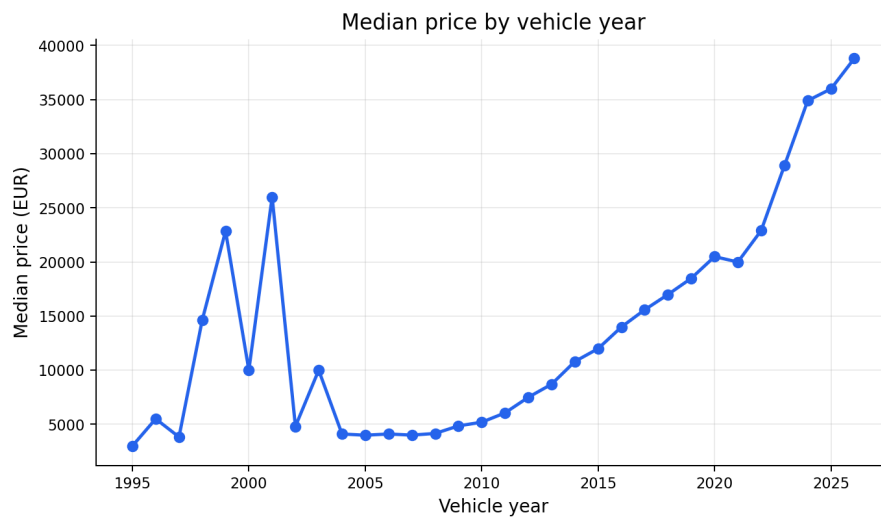


Figura 5.4: Mediana prețului în funcție de anul mașinii

Cele mai frecvente mărci sunt prezentate în Figura 5.5. Distribuția confirmă faptul că setul de date reflectă preferințele pieței locale, cu mărci comune în România și Europa.

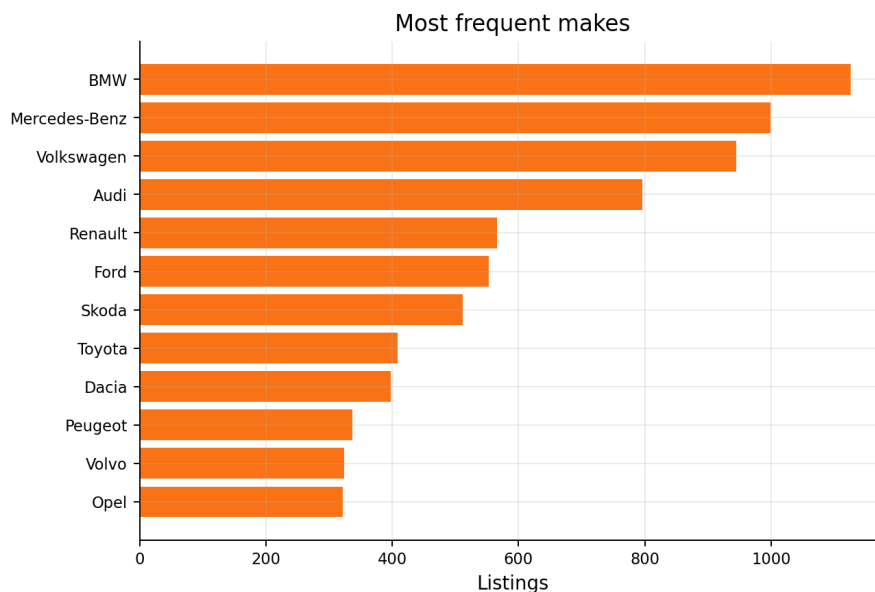


Figura 5.5: Cele mai frecvente mărci în setul de date

5.1.8 Pregătirea pentru antrenare

Setul final pentru antrenare nu include toate rândurile normalizate. Înainte de împărțirea train-test, pipeline-ul elimină exemple cu prețuri nerealiste, ani foarte vechi sau viitori nejustificați și valori tehnice extreme. Pentru antrenarea finală pe setul extins, raportul indică 9367 rânduri curate, dintre care 7493 au fost folosite la antrenare și 1874 la testare.

Caracteristicile candidate includ marca, modelul, combinația marcă-model, versiunea, anul, vârsta vehiculului, luna și anul primei înmatriculări, kilometrajul, kilometrajul pe an, combustibilul, transmisia, caroseria, puterea, capacitatea cilindrică, raportul cai putere pe litru, tipul vânzătorului și localizarea. Nu toate aceste caracteristici sunt păstrate. Selecția statistică este discutată în capitolul următor.

5.2 Modelare și selecția caracteristicilor

Componenta de modelare transformă datele curate în caracteristici relevante, elimină coloanele slab susținute statistic și compară algoritmi cu ipoteze diferite. În continuare sunt motivate etapele de preprocesare, alegerea modelelor, antrenarea pe log-preț și calculul estimării finale.

5.2.1 Caracteristici derivate

Modelele nu folosesc doar câmpurile extrase direct din anunțuri. Pipeline-ul construiește și caracteristici derivate care surprind relații mai utile pentru predicție. Vârsta vehiculului este calculată ca diferență între anul curent și anul mașinii. Kilometrajul pe an împarte kilometrajul total la vârsta vehiculului, evitând interpretarea identică a două mașini cu același kilometraj, dar cu vârste diferite. Raportul cai putere pe litru este calculat din putere și capacitate cilindrică,

oferind un indicator simplu al performanței specifice.

O caracteristică importantă este `make_model`, combinația dintre marcă și model. Aceasta ajută modelele să învețe că prețul nu depinde independent de marcă și model, ci de perechea lor. De exemplu, marca BMW are mai multe modele cu prețuri foarte diferite, iar modelul Seria 3 are altă distribuție decât X5. O reprezentare combinată reduce ambiguitatea.

Tabela 5.3: Exemple de caracteristici derivate

Caracteristică	Interpretare
<code>vehicle_age</code>	Vechimea mașinii în ani
<code>mileage_per_year</code>	Ritmul de utilizare anuală
<code>hp_per_liter</code>	Putere raportată la capacitatea cilindrică
<code>make_model</code>	Segment comercial aproximat prin marcă și model

5.2.2 Selecția statistică

Selecția caracteristicilor are rolul de a reduce zgomotul și de a elimina coloanele slab reprezentate. Pentru caracteristicile numerice, pipeline-ul calculează corelația Pearson față de prețul logaritmic. Pentru caracteristicile categorice, prețul este împărțit în intervale, iar asocierea dintre categorie și intervalul de preț este testată prin chi-pătrat. Mărimea efectului este exprimată printr-o valoare de tip Cramer V.

În configurația finală, pragurile au fost: $\alpha = 0.05$, corelație Pearson minimă absolută 0.05 pentru numerice și Cramer V minim 0.25 pentru categorice. Aceste praguri sunt suficient de permissive pentru a păstra semnale slabe, dar elimină câmpuri cu puține valori sau cu asociere insuficientă.

[Figura 5.6](#) prezintă cele mai relevante caracteristici. Printre cele păstrate se află versiunea, puterea, anul, vârsta vehiculului, combinația marcă-model, kilometrajul, capacitatea cilindrică, orașul și anul primei înmatriculări.

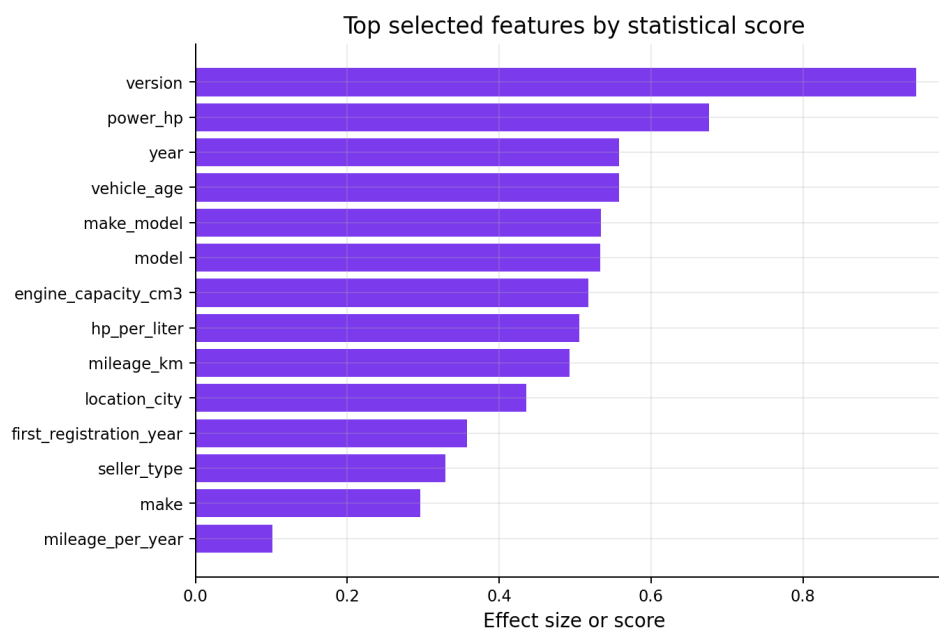


Figura 5.6: Caracteristici relevante după testele statistice

Tabela 5.4: Caracteristici selectate în antrenarea finală

Tip	Caracteristici
Numerice	year, vehicle_age, mileage_km, power_hp, engine_capacity_cm3
Numerice derivate	mileage_per_year, hp_per_liter
Categorice	make, model, make_model, version, seller_type, location_city
Înmatriculare	first_registration_year

Unele câmpuri au fost eliminate deși pot fi importante în realitate. `transmission` și `body_type` au fost eliminate în setul combinat deoarece erau completate pentru prea puține rânduri. `fuel_type` a avut semnificație statistică, dar efectul a fost sub pragul ales. Această situație indică o limitare a datelor, nu neapărat o lipsă de relevanță a combustibilului.

5.2.3 Preprocesare

Pipeline-ul folosește transformări diferite pentru variabile numerice și categorice. Variabilele numerice sunt imputate atunci când lipsesc și scalate pentru modelele sensibile la distanță, precum SVR și KNN. Variabilele categorice sunt imputate cu o valoare specială pentru lipsă și transformate prin one-hot encoding.

Această separare este importantă deoarece modelele au cerințe diferite. Arborii de decizie sunt mai puțin sensibili la scalare, dar SVR și KNN pot fi dominate de variabile cu valori mari, de exemplu kilometrajul. Prin standardizare, fiecare variabilă numerică are o contribuție comparabilă în spațiul de intrare.

5.2.4 Modele candidate și motivarea alegerii

Modelele au fost alese astfel încât experimentul să compare ipoteze diferite despre relația dintre caracteristicile mașinii și preț. Implementarea folosește scikit-learn [16], iar fiecare algoritm este inclus într-un pipeline care conține aceeași logică de imputare și codificare. Astfel, diferențele de performanță provin din model, nu dintr-o pregătire diferită a setului de test.

SVR cu kernel RBF

Support Vector Regression caută o funcție care tolerează erorile mici într-un tub de lățime ϵ și penalizează abaterile mai mari [22]. Kernelul RBF proiectează implicit datele într-un spațiu în care pot fi reprezentate relații neliniare. Am ales SVR deoarece prețul nu variază liniar cu anul sau kilometrajul: efectul unei diferențe de 20000 km depinde de vârstă, marcă și segment. Modelul oferă un contrast util față de arbori și față de regresia liniară, dar necesită standardizarea atentă a variabilelor numerice.

Ridge Regression

Ridge este o regresie liniară regularizată, în care coeficienții mari sunt penalizați. Am inclus-o ca model de referință simplu și stabil, mai ales după transformarea variabilelor categorice prin one-hot encoding. Dacă un model liniar obține rezultate apropiate de algoritmi mai complecși, înseamnă că o parte importantă a prețului poate fi explicată prin efecte aditive ale anului, kilometrajului și categoriilor. Regularizarea reduce riscul ca o marcă sau o versiune rară să primească un coeficient exagerat.

KNN cu ponderare după distanță

KNN estimează prețul folosind cele mai apropiate exemple din setul de antrenare, iar în varianta utilizată vecinii mai apropiați primesc o pondere mai mare. Alegerea este motivată de modul în care oamenii compară în practică mașinile: caută aceeași marcă și același model, apoi filtrează după an, kilometraj și motorizare. KNN oferă astfel o bază intuitivă pentru comparație. Dezavantajul este sensibilitatea la scalare și la zonele slab acoperite, unde cei mai apropiați vecini pot fi totuși foarte diferiți.

Random Forest

Random Forest construiește mai mulți arbori pe eșantioane și subseturi de caracteristici diferite, apoi mediază predicțiile [4]. L-am ales deoarece arborii surprind natural praguri și interacțiuni, de exemplu situația în care o putere mare crește prețul doar pentru anumite mărci sau generații. Ansamblul reduce variația unui arbore individual și este robust la relații neliniare, însă produce un artefact mare și nu extrapolează bine în afara intervalelor întâlnite la antrenare.

Extra Trees

Extra Trees folosește o randomizare mai puternică a pragurilor de separare decât Random Forest [8]. Motivul includerii sale este testarea efectului unei diversități mai mari între arbori asupra unui dataset zgomotos și eterogen. Randomizarea poate reduce varianța și poate evita

adaptarea excesivă la combinații accidentale de an, kilometraj și versiune, dar poate pierde precizie în segmentele rare ale pieței.

Gradient Boosting

Gradient Boosting construiește arborii secvențial, fiecare arbore nou încercând să corecteze reziduurile ansamblului existent [7]. Modelul a fost ales deoarece poate rafina treptat zonele în care o estimare inițială este insuficientă, de exemplu mașinile foarte noi sau foarte puternice. Spre deosebire de pădurile aleatoare, boosting-ul urmărește explicit erorile rămase, dar poate deveni sensibil la zgomot dacă este antrenat prea agresiv.

Voting Regressor

Voting Regressor combină predicțiile unor algoritmi cu erori diferite. A fost inclus pentru a verifica dacă SVR, Ridge și modelele de arbori se completează reciproc. Motivația este reducerea dependenței de o singură ipoteză: un model poate fi bun pentru mașini comune, iar altul pentru exemple premium. În experimentul offline, acest ansamblu a oferit cea mai bună performanță globală, în timp ce aplicația web folosește și o medie ponderată explicită, descrisă în Secțiunea 5.2.6.

La nivel de implementare, toate modelele urmează același flux general:

```
1 date = incarca_csv()
2 X, y = separa_caracteristici_si_pret(date)
3 y_log = log1p(y)
4
5 pentru fiecare model candidat:
6     pipeline = preprocesare + model
7     pipeline.fit(X_train, y_log_train)
8     predictii = expml(pipeline.predict(X_test))
9     calculeaza_RMSE_MAE_R2(predictii, y_test)
10
11 salveaza_modelul_cu_performanta_cea_mai_buna()
12 salveaza_metrice_grafice_si_predictii_individuale()
```

Listing 5.1: Pseudocod pentru antrenarea modelelor

Această structură comună a permis compararea corectă a modelelor. Diferența dintre ele nu stă în datele primite sau în transformările aplicate, ci în modul în care fiecare model învață relația dintre caracteristici și preț.

5.2.5 Antrenarea pe log-preț

Primele experimente au arătat că modelele pot produce predicții prea apropiate între anunțuri diferite, mai ales când distribuția țintei este dezechilibrată. Pentru a corecta parțial problema, modelele finale au fost antrenate pe $\log_1p(\text{price})$. Această transformare reduce presiunea exemplorilor foarte scumpe și face ca erorile relative să conteze mai echilibrat.

În aplicație, predicțiile sunt inversate automat în EUR înainte de afișare. Utilizatorul nu vede valorile logaritmice. El vede prețul estimat, prețul anunțului și diferența procentuală.

5.2.6 Predicția finală prin medie ponderată

Aplicația nu este limitată la un singur model. Pentru fiecare anunț, sunt calculate predicții pentru modelele salvate în bundle. Predicția finală afișată utilizatorului este o medie ponderată:

$$\hat{p}_{final} = \frac{\sum_i w_i \hat{p}_i}{\sum_i w_i}$$

Prima strategie disponibilă, `inverse_mae`, folosește ponderi invers proporționale cu MAE-ul modelului:

$$w_i = \frac{1}{MAE_i + \varepsilon}$$

A doua strategie, `inverse_mae_with_agreement`, adaugă o penalizare pentru modelele care se abat mult de la mediana predicțiilor:

$$w_i = \frac{1}{MAE_i + \varepsilon} \cdot \exp\left(-\frac{|\hat{p}_i - \text{median}(\hat{p})|}{s}\right)$$

Această a doua strategie a fost introdusă deoarece un model poate avea performanță bună global, dar poate produce o predicție foarte diferită pentru un anunț atipic. Penalizarea de acord nu elimină modelul, dar îi reduce influența în predicția finală.

Voting Regressor este evaluat și afișat ca model individual, însă nu primește o pondere separată în această medie. El combină deja o parte dintre modelele de bază, iar includerea sa ar număra din nou aceleași predicții. În interfață, câmpurile de pondere și acord sunt marcate explicit ca neaplicabile pentru acest model.

5.2.7 Verdictul

Verdictul compară prețul din anunț cu prețul estimat. Dacă diferența procentuală absolută este sub pragul ales de utilizator, anunțul este considerat corect. Dacă prețul publicat este mult sub estimare, verdictul devine preț prea mic, cu interpretare de ofertă potențial suspectă. Dacă prețul publicat este mult peste estimare, verdictul devine preț prea mare.

Pragul implicit este 15%, dar interfața permite modificarea lui. Această opțiune este necesară deoarece toleranța diferă în funcție de buget și categorie. Pentru o mașină ieftină, 10% poate fi o diferență importantă. Pentru o mașină premium, aceeași toleranță relativă poate corespunde unei sume mari, dar poate fi încă acceptabilă în negociere.

5.3 Implementarea backend-ului

Backend-ul pornește o aplicație FastAPI și montează folderul interfeței ca resurse statice. Modulele Pydantic validează automat tipurile și limitele parametrilor. De exemplu, toleranța trebuie să fie pozitivă și cel mult 100, iar metoda de agregare trebuie să fie una dintre opțiunile implementate.

[Fragmentul 5.2](#) arată partea centrală a endpoint-ului de predicție. Ruta nu conține logica matematică, ci rezolvă căile configurate, verifică artefactele și apelează serviciul de inferență. Această separare face posibilă folosirea aceleiași funcții și din CLI.

```

1 @app.post("/predict")
2 def predict(request: PredictRequest,
3             user: AuthUser = Depends(get_current_user)):
4     model_bundle = resolve_model_bundle_path()
5     similarity_csv = resolve_similarity_csv_path()
6
7     prediction = predict_from_link(
8         site=request.site,
9         url=request.url,
10        model_bundle_path=model_bundle,
11        threshold_percent=request.threshold_percent,
12        similarity_csv_path=similarity_csv,
13        ensemble_method=request.ensemble_method,
14    )
15
16    payload = prediction.to_dict()
17    record_prediction_history(user_id=user.id,
18                           prediction=payload)
19    return payload

```

Listing 5.2: Orchestrarea predicției în endpoint-ul FastAPI

Funcția `predict_from_link` validează domeniul și selectează fluxul Autovit sau mobile.de. Rezultatul este reprezentat printr-o structură care conține atât verdictul, cât și datele intermediare necesare interfeței. Excepțiile de validare devin răspunsuri 400, iar erorile sursei externe sunt transformate în mesaje controlate.

5.4 Implementarea frontend-ului

Frontend-ul folosește un formular HTML și o singură cerere asincronă către `/predict`. Înaintea trimerii sunt validate domeniul linkului și toleranța. Pe durata cererii, butonul este dezactivat și este afișat overlay-ul de încărcare. Blocul `finally` este important deoarece închide animația atât la succes, cât și la eroare.

```

1 setLoading(true) ;

```

```
2 try {
3   const response = await fetch("/predict", {
4     method: "POST",
5     credentials: "same-origin",
6     headers: {"Content-Type": "application/json"},
7     body: JSON.stringify({
8       site,
9       url,
10      threshold_percent: thresholdPercent,
11      ensemble_method: ensembleMethod,
12      similar_limit: 4,
13    }),
14  });
15
16  const data = await response.json();
17  if (!response.ok) {
18    throw new Error(data.detail);
19  }
20  renderResult(data);
21 } catch (error) {
22   setMessage(error.message, "error");
23 } finally {
24   setLoading(false);
25 }
```

Listing 5.3: Cererea frontend către API și închiderea stării de încărcare

Rezultatul este construit din carduri: informațiile vehiculului, verdictul, predicțiile pe model, graficele și anunțurile similare. Estimarea finală ponderată și verdictul sunt afișate imediat, iar estimările individuale și graficele sunt plasate într-un element HTML `details`, închis implicit. Această afișare progresivă păstrează concluzia ușor de citit și permite extinderea detaliilor numai când sunt necesare.

Frontend-ul tratează cele trei adrese `/`, `/istoric` și `/explicatii` ca pagini distincte, deși FastAPI servește același document HTML. JavaScript-ul activează secțiunea corespunzătoare adresei și marchează legătura curentă în bara de navigare. Pagina de explicații definește pe înțelesul utilizatorului metricile și regulile verdictului. Aceleași texte sunt disponibile printr-un element `dialog` deschis de butoanele de ajutor contextual.

Nu este folosit HTML primit de la platformele externe; valorile sunt introduse în șabloane controlate de aplicație și sunt escapate înainte de afișare. Preferința vizuală este memorată în `localStorage`, iar layout-ul folosește `grid` și `flexbox` pentru adaptarea cardurilor la desktop și mobil.

Figura 5.7 prezintă interfața după analiza unui anunț. În aceeași zonă sunt vizibile linkul,

strategia de agregare, toleranța, confirmarea operației și cardul rezultatului. Fotografia ajută la verificarea identității anunțului, însă nu este folosită ca intrare pentru modelele curente.

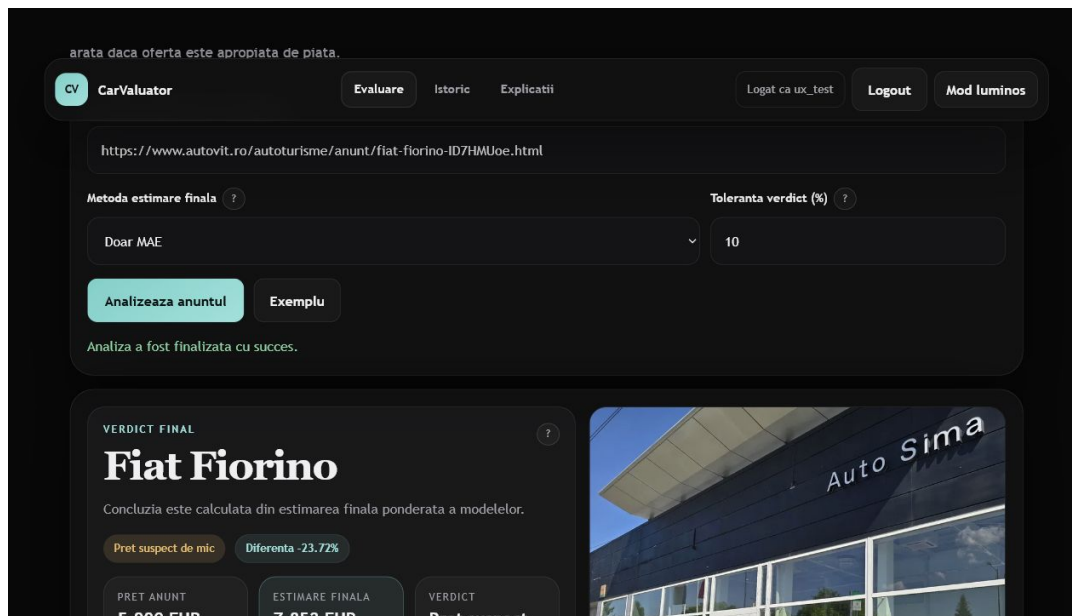


Figura 5.7: Interfața web după finalizarea unei predicții

5.5 Implementarea autentificării și securității

Parola este procesată cu PBKDF2-HMAC-SHA256. Pentru fiecare cont este generat un salt nou, iar baza păstrează algoritmul, numărul de iterații, saltul și hash-ul într-un șir serializat. Verificarea reconstruiește hash-ul și folosește `hmac.compare_digest`, evitând comparațiile caracter cu caracter.

```

1 salt = secrets.token_bytes(16)
2 derived = hashlib.pbkdf2_hmac(
3     "sha256",
4     password.encode("utf-8"),
5     salt,
6     PASSWORD_ITERATIONS,
7 )
8 password_hash = (
9     f"pbkdf2_sha256${PASSWORD_ITERATIONS}$"
10    f"${salt.hex()}${derived.hex()}"
11 )

```

Listing 5.4: Principiul de derivare sigură a parolei

Tokenul de sesiune este generat aleator și stocat în tabela `sessions`. Cookie-ul este HTTP-only, astfel încât JavaScript-ul paginii nu poate citi direct tokenul. Rutele de istoric pri-

mesc utilizatorul curent printr-o dependență FastAPI, iar comenzile SQL includ întotdeauna `user_id`; un utilizator nu poate accesa sau șterge înregistrările altui cont. Endpoint-ul `GET /history/{id}` citește JSON-ul salvat și returnează raportul complet numai dacă perechea dintre identificator și utilizator este validă.

Validarea URL-urilor acceptă numai domeniile prevăzute. Aplicația nu execută cod din paginile analizate și nu expune în interfață conținut HTML brut. Pentru deployment, cookie-ul este marcat secure, iar toate cererile publice sunt transportate prin HTTPS-ul oferit de platformă.

5.5.1 Limitarea ratei cererilor

Limitatorul folosește câte o coadă cu momentele cererilor pentru fiecare pereche formată din scop și cheie. La o cerere nouă sunt eliminate momentele mai vechi decât fereastra configurată. Dacă numărul rămas a atins limita, cererea este respinsă înainte de hashing-ul parolei, scraping sau inferență. Accesul la structuri este protejat cu un lock deoarece endpoint-urile sincrone FastAPI pot rula concurrent în thread-uri diferite.

```
1 decision = limiter.check(  
2     scope="predict",  
3     key=str(user.id),  
4     policy=settings.predict_policy,  
5 )  
6 if not decision.allowed:  
7     raise HTTPException(  
8         status_code=429,  
9         detail="Prea multe cereri de analiza.",  
10        headers={"Retry-After": str(decision.retry_after_seconds)}),  
11    )
```

Listing 5.5: Verificarea unei limite cu fereastră glisantă

Configurația implicită permite 60 de cereri pe minut pentru un IP, 10 încercări de login în 15 minute, 3 înregistrări pe oră și 5 predicții pe minut pentru fiecare cont. Antetele `X-RateLimit-Limit`, `X-RateLimit-Remaining` și `Retry-After` fac comportamentul observabil. Ruta `/health` expune valorile active, fără a expune contoarele sau identitatea utilizatorilor.

5.6 Implementarea similarității

Motorul de similaritate pornește de la aceeași reprezentare normalizată ca modelele. Sunt favorizate aceeași marcă și același model, apoi se calculează distanțe pentru an, kilometraj, putere și capacitate cilindrică. Combustibilul și transmisia adaugă bonusuri atunci când coincid.

Prețurile anunțurilor similare nu modifică predicția finală. Această decizie evită dublarea informației, deoarece modelele au fost deja antrenate pe aceleași tipuri de exemple. Lista are rol explicativ și de navigare, iar scorul său este separat de ponderile ensemble-ului.

Datasetul de similaritate este tratat ca un snapshot operațional distinct de setul de antrenare. Scriptul dedicat de actualizare colectează periodic anunțuri curente, le normalizează și actualizează fișierul stabil `similarity_current.csv`. Separarea permite reducerea linkurilor expirate fără reantrenarea modelelor și fără modificarea estimărilor de preț.

5.7 Artefacte și deployment pe Render

Antrenarea locală produce bundle-ul complet, metricile CSV, raportul JSON și graficele PNG. Scriptul PowerShell pentru pregătirea artefactelor Render creează o variantă redusă pentru cloud și copiază datasetul de similaritate. Fișierul `render.yaml` declară serviciul Docker, iar `Dockerfile` instalează Python 3.11, dependențele fixate și Chromium.

Fixarea versiunilor este necesară deoarece obiectele `joblib` folosesc clase interne din `scikit-learn`. Un model serializat cu versiunea 1.8.0 poate eșua la încărcare într-un runtime diferit. Containerul elimină această diferență și rezolvă și dependența `Playwright` de un browser instalat.

Render rulează aceeași aplicație, dar cu un bundle redus pentru limita de memorie a planului gratuit. Datasetul de similaritate este copiat separat în subdirectorul `datasets` al folderului `deploy_artifacts`, sub numele `similarity_current.csv`. Un refresh efectuat în folderul local `data` nu afectează automat site-ul public; artefactul trebuie regenerat, publicat în Git și urmat de redeploy. Accesul live la `mobile.de` rămâne dependent de politica platformei față de IP-urile de cloud; de aceea, lookup-ul în snapshot-ul livrat precedă scraping-ul live.

Capitolul 6

Testarea aplicației și rezultate

6.1 Rolul studiilor de caz

Evaluarea numerică a modelelor este necesară, dar nu este suficientă pentru un proiect care trebuie folosit printr-o interfață web. Un model poate avea scoruri bune pe setul de test, dar aplicația poate eșua dacă nu extrage corect linkul, dacă afișează imaginea greșită, dacă pierde sesiunea utilizatorului sau dacă istoricul devine greu de folosit. Din acest motiv, proiectul a fost validat și prin scenarii funcționale.

Studiile de caz urmăresc comportamentul complet al aplicației, de la introducerea linkului până la salvarea rezultatului în cont. Ele nu sunt prezentate ca benchmark statistic, ci ca verificare a experienței reale. Pentru un cumpărător, aplicația contează doar dacă răspunde rapid, afișează informații inteligibile și nu ascunde situațiile în care predicția este incertă.

6.2 Scenariu Autovit cu mașină comună

Primul scenariu folosește un anunț Autovit pentru o mașină comună, de exemplu un model compact sau de clasă medie, cu an și kilometraj tipice pentru piața românească. În acest caz, aplicația are cele mai bune șanse să producă o estimare utilă, deoarece setul de antrenare conține multe exemple asemănătoare.

Fluxul observat este următorul. Utilizatorul se autentifică, introduce linkul, alege metoda implicită `inverse_mae_with_agreement` și păstrează toleranța de 15%. Interfața afișează animația de încărcare, backend-ul extrage pagina, normalizează câmpurile, încarcă modelul și returnează rezultatul. Cardul rezultatului conține titlul, prețul publicat, prețul estimat, verdictul și diferența procentuală.

Pentru mașini comune, lista de anunțuri similare este de obicei relevantă. Aplicația găsește mașini cu aceeași marcă și același model, apoi prioritizează valori apropiate pentru an și kilometraj. Acest lucru este important deoarece utilizatorul poate verifica vizual dacă verdictul este plauzibil. Dacă prețul estimat este 12000 EUR, iar anunțurile similare sunt între 11000 și

13000 EUR, verdictul „preț corect” este ușor de înțeles.

În acest scenariu, toate componentele au rol vizibil. Modelul produce estimarea, dar sugestiile similare oferă context. Imaginea anunțului confirmă că linkul a fost interpretat corect. Istoricul salvează analiza, astfel încât utilizatorul poate compara ulterior mai multe mașini.

6.3 Scenariu Autovit cu mașină premium

Al doilea scenariu folosește o mașină premium sau foarte scumpă. Acesta este un caz mai dificil deoarece distribuția prețurilor este asimetrică, iar exemplele premium sunt mai puține. În graficele de evaluare se observă că unele mașini scumpe sunt subestimate. Cauza nu este doar modelul, ci și faptul că prețul premium depinde de detalii pe care scraperul nu le captează complet: pachet de dotări, istoric de service, ediție limitată, stare estetică, garanție sau raritate.

Pentru acest scenariu, afișarea predicțiilor pe model este esențială. Dacă SVR, Ridge și ansamblul de votare sunt apropiate, iar arborii sunt mai departe, utilizatorul poate avea o încredere mai mare în estimarea centrală. Dacă predicțiile diferă mult, aplicația nu ar trebui interpretată ca verdict definitiv. Din acest motiv, strategia `inverse_mae_with_agreement` este utilă: modelele care se abat mult de la mediana predicțiilor primesc o pondere mai mică.

În cazul mașinilor premium, toleranța de 15% poate însemna o sumă mare. Utilizatorul poate reduce pragul la 10% dacă dorește o evaluare mai strictă. Această opțiune a fost adăugată tocmai pentru a evita un verdict rigid. O aplicație de evaluare trebuie să respecte faptul că noțiunea de „preț corect” depinde de context.

6.4 Scenariu mobile.de

Scenariul mobile.de validează integrarea cea mai fragilă. Utilizatorul introduce un link localizat către pagina de detaliu, împreună cu identificatorul numeric al anunțului. Backend-ul trebuie să extragă identificatorul și să parseze pagina. În primele iterații, astfel de linkuri nu funcționau deoarece parserul se aștepta la forme diferite ale URL-ului.

După extindere, aplicația poate trata linkuri de detaliu mobile.de, însă succesul depinde de răspunsul site-ului. Dacă pagina returnează o eroare de acces, aplicația afișează o eroare de predicție. Dacă pagina este disponibilă, câmpurile sunt extrase și normalizate similar cu Autovit.

O problemă observată în acest scenariu a fost imaginea greșită. Pentru anumite anunțuri, prima imagine accesibilă era un banner de dealer sau o fotografie generală de showroom. În urma testării manuale, parserul a fost ajustat pentru a prefera imaginile din galeria vehiculului. Această corecție este relevantă pentru calitatea aplicației deoarece imaginea este un semnal vizual puternic: dacă imaginea este greșită, se poate crea impresia că întregul anunț a fost analizat greșit.

6.5 Scenariu de istoric pe cont

Autentificarea și istoricul au fost adăugate deoarece utilizatorii testează mai multe anunțuri într-o sesiune. Fără istoric, fiecare analiză dispare după refresh. Cu istoric, aplicația devine un instrument de lucru: utilizatorul poate compara oferte, poate reveni la o analiză și poate elimina testele irelevante.

Validarea acestui scenariu urmărește patru operații. Prima este crearea contului și autentificarea prin cookie HTTP-only. A doua este salvarea automată a unei predicții reușite. A treia este redeschiderea raportului complet din pagina dedicată istoricului. Verificarea confirmă păstrarea toleranței, a metodei de agregare, a celor patru estimări individuale și a celor patru sugestii similare folosite în raportul inițial. A patra este ștergerea unui element individual sau a întregului istoric. Această ultimă funcționalitate păstrează aplicația utilizabilă atunci când sunt comparate multe anunțuri într-o singură sesiune.

Din perspectivă de proiectare, istoricul este legat strict de contul curent. Un utilizator nu poate șterge sau vedea analizele altui utilizator. Această regulă este verificată în backend prin dependența de autentificare înainte de accesarea rutelor `/history`.

6.6 Scenariu de deployment

Deployment-ul a fost pregătit pentru Render. Scenariul de validare pornește de la o problemă practică: folderul `data` este ignorat de Git, dar aplicația are nevoie de model și CSV pentru a funcționa în cloud. Soluția a fost introducerea folderului `deploy_artifacts`, care conține un model comprimat, datasetul de similaritate și graficele de performanță.

Acest scenariu verifică ruta `/health`. Dacă modelul și CSV-ul există, ruta întoarce status „ok” și indică faptul că graficele pot fi servite. În testarea locală cu variabilele Render, ruta a confirmat existența modelului, a datasetului și a graficelor principale. Această verificare este importantă deoarece multe erori de deployment nu apar în cod, ci în calea fișierelor sau în variabilele de mediu.

O limitare a deployment-ului demo este baza de date de autentificare. Dacă este plasată în `/tmp`, conturile pot fi pierdute la restart. Pentru un serviciu public stabil, este necesar un disc persistent sau o bază de date externă. Lucrarea marchează această diferență explicit, pentru ca demo-ul și producția să nu fie confundate.

6.7 Testarea limitării ratei

Prima observație preliminară primită de la un coleg înaintea sesiunii extinse de testare a utilizabilității a fost necesitatea unui mecanism de limitare a ratei. Observația este relevantă deoarece analiza unui link poate porni scraping și mai multe modele ML, iar autentificarea implică o funcție PBKDF2 intenționat costisitoare. Fără limitare, un script simplu ar putea consuma memoria și timpul de procesor al instanței gratuite sau ar putea încerca repetat parole.

În urma observației a fost introdusă o fereastră glisantă configurabilă. Testele unitare verifică

blocarea după atingerea pragului, eliberarea limitei după expirarea ferestrei, separarea adreselor IP și separarea conturilor. Un test HTTP a configurat temporar limita de login la două cereri pe minut: primele două autentificări cu date greșite au răspuns 401, iar a treia a răspuns 429 și a inclus `Retry-After: 60`. Astfel, mecanismul nu ascunde erorile normale de autentificare, dar oprește repetarea lor excesivă.

Pentru sesiunea cu colegii, valorile implicite rămân suficient de permissive pentru utilizare normală: fiecare cont poate face cinci analize pe minut, iar limita generală este de 60 de cereri pe minut pentru un IP. Dacă mai mulți participanți folosesc aceeași rețea, predicțiile sunt în continuare separate după cont; limita generală rămâne o protecție de ultim nivel. Contoarele sunt în memorie și se resetează la restart, limitare acceptabilă pentru un demo cu o singură instanță.

6.8 Validarea mesajelor de eroare

O aplicație care depinde de scraping trebuie să trateze erorile ca parte normală a fluxului. Linkurile pot expira, site-urile pot bloca cererea, modelul poate lipsi, iar datasetul de similaritate poate să nu fie configurat. În loc să se blocheze, aplicația întoarce mesaje explicite.

Pentru utilizator, cel mai frecvent mesaj este eșecul predicției din cauza conexiunii sau a site-ului țintă. Într-un astfel de caz, mesajul trebuie să indice problema fără a expune detalii inutile. Pentru dezvoltator, ruta `/health` ajută la diagnosticarea rapidă a configurării.

Această abordare este importantă pentru un proiect real. Un sistem ML integrat într-o aplicație web nu este evaluat doar prin scoruri, ci și prin robustețe operațională. Dacă aplicația explică de ce nu poate analiza un link, utilizatorul are mai multă încredere decât într-o aplicație care eșuează tăcut.

6.9 Observații din testare

Testarea manuală și feedback-ul colegilor au scos la iveală probleme care nu erau evidente din metricele offline. Prima problemă a fost uniformizarea predicțiilor unor modele. A doua a fost imaginea greșită pe `mobile.de`. A treia a fost aglomerarea istoricului și pierderea contextului atunci când era păstrat doar un rezumat. A patra a fost nevoia ca utilizatorul să aleagă metoda de combinare și toleranța. Au fost cerute și explicații mai accesibile pentru termenii statistici, precum și o separare mai clară între concluzia principală și detaliile modelelor. Observația preliminară de utilizabilitate privind limitarea ratei a adăugat și o cerință operațională care nu era vizibilă în evaluarea modelelor.

Aceste observații arată valoarea iterațiilor cu interfața reală. Pipeline-ul ML poate părea complet într-un notebook, dar aplicația finală expune fricțiuni pe care doar utilizarea efectivă le dezvăluie. În proiectul `CarValuator`, fiecare observație a dus la o modificare concretă: log-preț, medie ponderată, filtrare de imagini, rapoarte complete în istoric, explicații contextuale, afișare progresivă, ajustări generale de UX și protecție împotriva cererilor excesive.

6.10 Concluziile studiilor de caz

Studiile de caz confirmă că aplicația este funcțională pentru fluxul principal Autovit și utilizabilă pentru anumite linkuri mobile.de, cu limite clare. Ele confirmă și importanța separării între evaluarea modelului și evaluarea aplicației. Un scor bun nu garantează o experiență bună, iar o interfață bună nu compensează date incomplete sau blocaje ale sursei externe.

Prin aceste scenarii, proiectul a fost ajustat pentru a fi mai aproape de o aplicație reală. Nu este doar un script care prezice prețuri, ci un sistem cu utilizatori, istoric, erori, grafice, deployment și decizii explicite de proiectare.

6.11 Evaluare experimentală

Evaluarea experimentală reunește măsurarea performanței modelelor, analiza erorilor și verificarea comportamentului aplicației pe anunțuri reale. Rezultatele sunt interpretate atât statistic, cât și prin comparație cu indicatorii oferiți de platformele studiate.

6.11.1 Configurația experimentului

Evaluarea finală a fost realizată pe setul combinat extins, format din 9633 de anunțuri. După curățarea finală aplicată de pipeline-ul de antrenare, au rămas 9367 exemple. Împărțirea train-test a folosit 80% pentru antrenare și 20% pentru testare, rezultând 7493 rânduri de antrenare și 1874 rânduri de test.

Toate modelele au fost antrenate pe ținta logaritmică a prețului. Predicțiile au fost convertite înapoi în EUR înainte de calcularea metricilor finale. Această alegere face rezultatele mai ușor de interpretat și permite compararea directă cu prețurile reale ale anunțurilor.

6.11.2 Rezultatele modelelor

[Tabelul 6.1](#) prezintă rezultatele principale. Cel mai bun rezultat global a fost obținut de ansamblul de votare, cu RMSE de 4602 EUR, MAE de 2696 EUR și $R^2 = 0.9231$. SVR RBF a rămas competitiv, cu $R^2 = 0.9158$, iar Extra Trees a obținut $R^2 = 0.9095$. Față de experimentul anterior, R^2 maxim este ușor mai mic, dar testul este mai realist deoarece include un volum mult mai mare de anunțuri mobile.de și o distribuție mai eterogenă.

Tabela 6.1: Performanța modelelor pe setul de test

Model	RMSE	MAE	R^2
Voting Ensemble	4602.16	2695.83	0.9231
SVR RBF	4816.44	2729.41	0.9158
Extra Trees	4991.76	2967.41	0.9095
Gradient Boosting	5093.28	3215.95	0.9058
Random Forest	5292.42	3175.31	0.8983
KNN Distance	5660.18	3319.93	0.8837
Ridge	5774.39	2924.75	0.8789

[Figura 6.1](#) prezintă comparația vizuală a performanței modelelor, iar [Figura 6.2](#) pune în paralel MAE și R^2 .

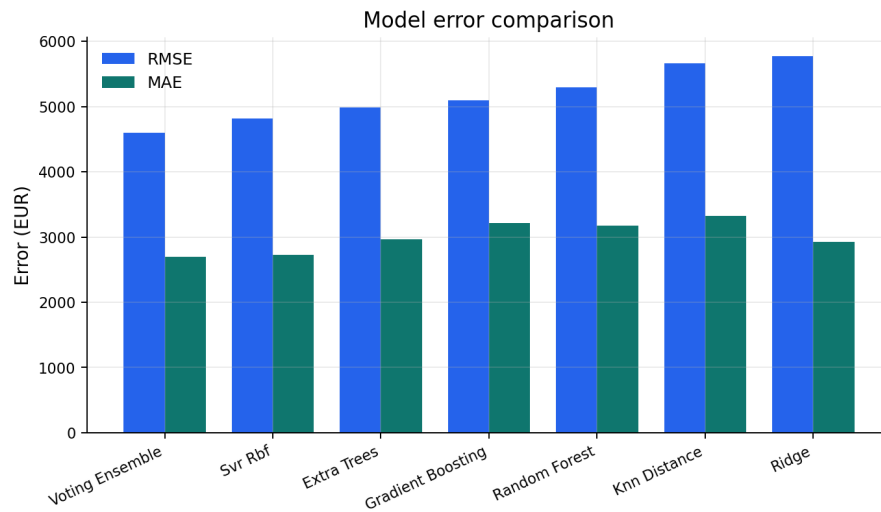
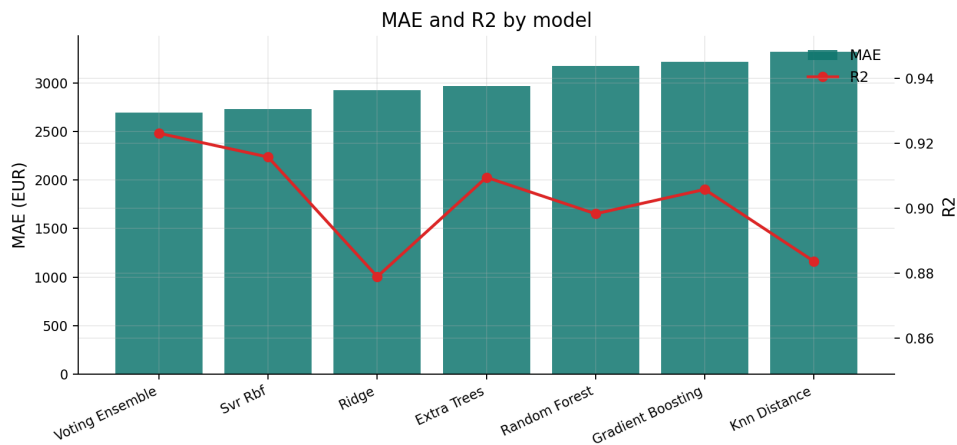


Figura 6.1: Comparație de performanță între modele

Figura 6.2: MAE și R^2 pentru modelele evaluate

Rezultatele arată că ansamblul de votare oferă cea mai bună stabilitate pe setul extins. SVR RBF rămâne foarte bun, dar modelele de arbori contribuie la reducerea erorii atunci când sunt combinate cu modele sensibile la distanță. Ridge are în continuare MAE competitiv, dar R^2 mai slab, semn că relațiile din setul extins sunt mai puțin liniare decât în experimentul dominat de Autovit.

6.11.3 Analiza predicțiilor

Figura 6.3 arată relația dintre prețurile reale și cele estimate de modelul ales. Punctele sunt concentrate în jurul diagonalei, ceea ce indică predicții rezonabile pentru majoritatea anunțurilor. Totuși, se observă și cazuri de subestimare pentru vehicule foarte scumpe, în special modele premium sau configurații rare.

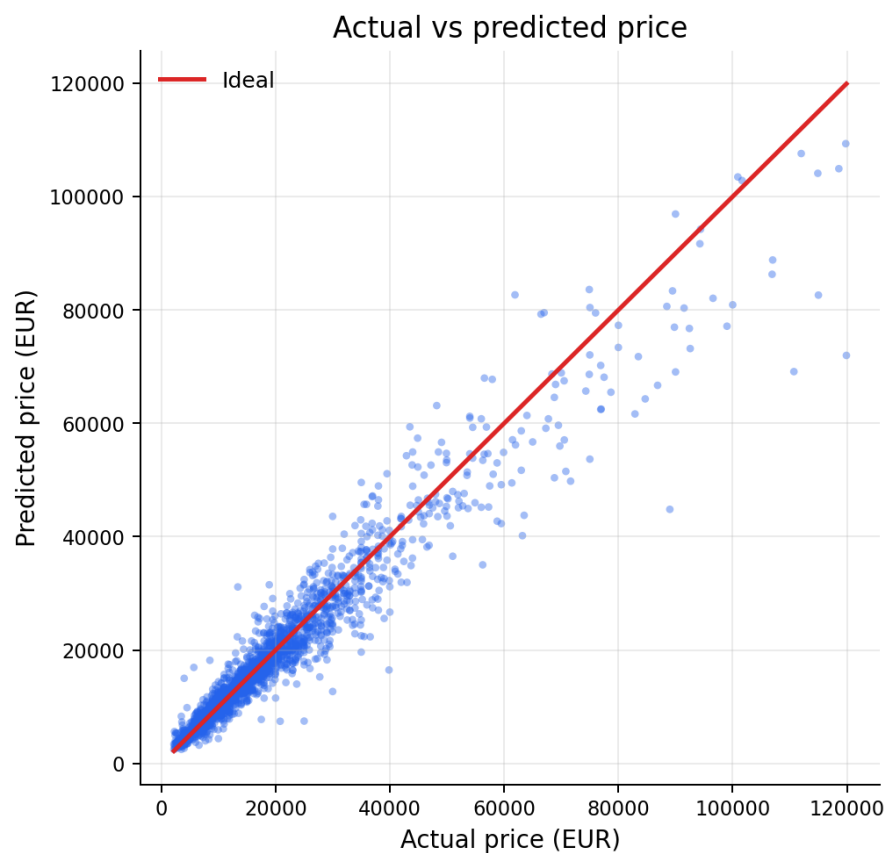


Figura 6.3: Preț real față de preț estimat

Figura 6.4 prezintă distribuția reziduurilor și reziduu în funcție de prețul real. Distribuția are centru apropiat de zero, dar cozi vizibile. Cozile reprezintă anunțuri unde modelul greșește semnificativ, de obicei din cauza informațiilor neobservate: echipări speciale, stare tehnică, istoric, raritate sau poziționare comercială atipică.

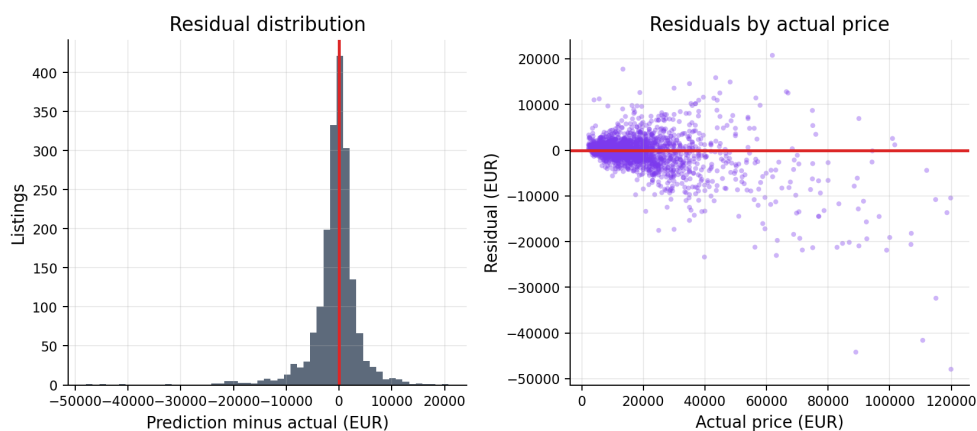


Figura 6.4: Analiza reziduurilor pentru modelul ales

6.11.4 Corelații și interpretabilitate

Matricea de corelații din Figura 6.5 confirmă câteva relații așteptate. Anul și vârsta vehiculului sunt puternic legate de preț. Kilometrajul este corelat negativ cu prețul, iar puterea are corelație pozitivă. Capacitatea cilindrică și puterea sunt de asemenea legate între ele.

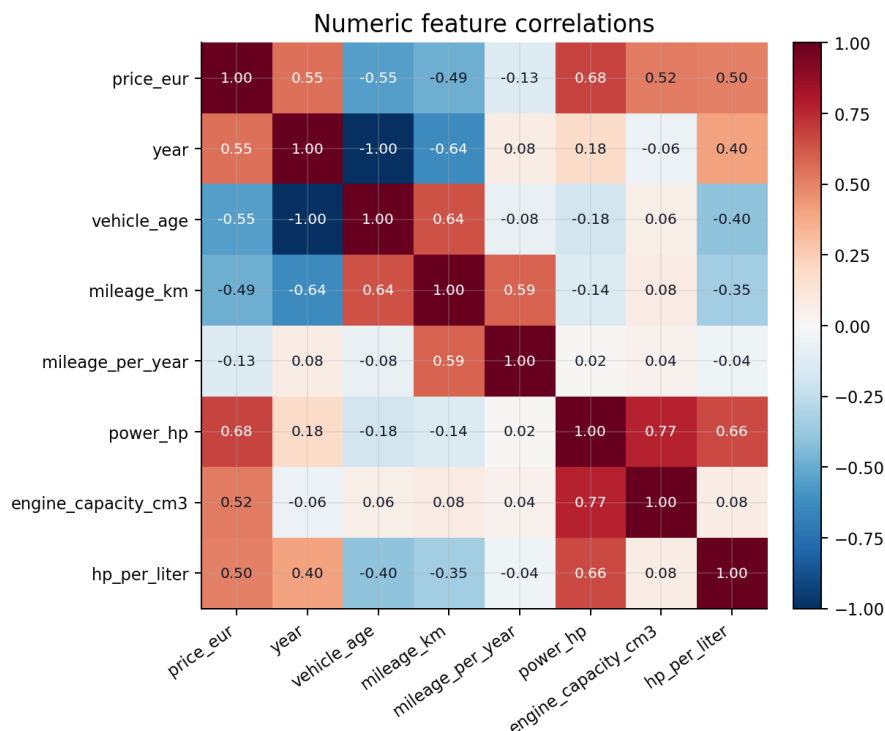


Figura 6.5: Matrice de corelații pentru caracteristici numerice

Interpretabilitatea este limitată de folosirea one-hot encoding și de modelele neliniare. Totuși, selecția statistică oferă un nivel de explicație înainte de antrenare. Utilizatorul final nu vede direct importanța caracteristicilor, dar aplicația îi arată anunțuri similare, ceea ce face verdictul mai ușor de verificat intuitiv.

6.11.5 Testare prin aplicație

Testarea aplicației a inclus linkuri Autovit și mobile.de. Pentru Autovit, fluxul a fost în general stabil: linkul este analizat, imaginea principală apare corect, iar rezultatul este salvat în istoric. Pentru mobile.de, fluxul a necesitat mai multe ajustări. Anumite linkuri de detaliu nu erau recunoscute inițial deoarece forma URL-ului diferă între limbi și pagini. Ulterior, integrarea a fost extinsă pentru linkuri de tip `/ro/vehicule/detalii.html?id=...`

O observație importantă din testarea manuală a fost că unele modele produceau predicții foarte apropiate pentru mașini foarte diferite. Această problemă a fost vizibilă mai ales la SVR și KNN în primele variante de model. Cauzele posibile au fost scalarea, datele insuficiente pentru anumite zone de preț și distribuția dezechilibrată a țintei. Antrenarea pe log-preț și folosirea

unui set mai mare au redus această problemă, dar nu au eliminat complet riscul pentru anunțuri extreme.

6.11.6 Comparatie cu indicatorii platformelor

Pentru a verifica dacă rezultatele au sens practic, am comparat câteva exemple din setul de test cu etichetele disponibile în datele sursă. Etichetele platformelor au fost normalizate în trei clase: *fair*, *low* și *high*. Ele nu sunt perfect echivalente cu verdictul CarValuator, deoarece fiecare platformă folosește propriile reguli, propriile seturi de comparație și propriile praguri. Totuși, comparația este utilă ca verificare de plauzibilitate.

Tabela 6.2: Exemple de comparație între estimarea CarValuator și etichetele platformelor

Sursă	Anunț	Listat	Estimat	Dif.	Platformă	CarValuator
Autovit	Toyota Hilux 4x4 Double Cab M/T Comfort	26250	26249	0.0%	fair	preț corect
Autovit	Audi A7 Sportback	32900	36558	-10.0%	low	preț corect
Autovit	Renault Clio V TCe Zen	9900	8998	10.0%	high	preț corect
mobile.de	Volkswagen Touareg 3.0 V6 4M.	21790	22277	-2.2%	fair	preț corect
mobile.de	BMW M135i xDrive	19980	21708	-8.0%	low	preț corect
mobile.de	Volkswagen Golf VII Variant Comfortline	12690	10342	22.7%	high	prea mare

Exemplele arată că aplicația se comportă coerent în cazurile unde diferența este mare. Pentru Volkswagen Golf VII, prețul listat este cu aproximativ 22.7% peste estimare, deci verdictul CarValuator devine „prea mare”, în acord cu eticheta *high*. Pentru exemplele cu diferențe de aproximativ 8-10%, verdictul rămâne „preț corect” deoarece pragul implicit este 15%. Această diferență nu este o contradicție, ci rezultatul pragului configurabil: dacă utilizatorul alege 10%, unele cazuri marginale pot trece în zona „prea mic” sau „prea mare”.

6.11.7 Calitatea setului de date

Calitatea datelor este principalul factor care limitează precizia. Pentru Autovit, multe câmpuri importante lipsesc sau sunt neuniforme. De exemplu, transmisia și caroseria nu au fost suficient de complete în setul folosit. Pentru mobile.de, volumul de exemple este mult mai bun după metoda Markdown, iar puterea este extrasă pentru majoritatea rândurilor. Capacitatea cilindrică a fost recuperată parțial din titlurile care includ explicit valori precum 2.0 TDI sau 1.5 TSI, însă transmisia, caroseria și o parte din câmpurile tehnice lipsesc frecvent. În plus, datele provin din momente diferite și pot reflecta condiții temporare ale pieței.

Modelele învață prețurile publicate, nu prețurile finale de tranzacție. Această diferență este esențială. Un anunț poate avea preț negociabil, iar prețul real de vânzare poate fi mai mic. Prin urmare, CarValuator estimează corectitudinea față de piața anunțurilor, nu față de tranzacțiile efective.

6.11.8 Acuratețe și utilitate practică

Un MAE de aproximativ 2700 EUR pentru cel mai bun model după această metrică este rezonabil pentru un prototip bazat pe scraping public, dar nu suficient pentru decizii financiare fără verificare suplimentară. Utilitatea aplicației este mai mare ca instrument de filtrare preliminară. Ea poate identifica rapid anunțuri care merită investigate sau anunțuri care par mult în afara pieței.

Pentru mașini comune, cu multe exemple în setul de date, estimările sunt mai credibile. Pentru mașini rare, modele premium, vehicule modificate sau anunțuri cu dotări speciale, eroarea poate crește. De aceea, interfața afișează și predicțiile individuale ale modelelor, nu doar media finală. Dacă modelele nu sunt de acord, utilizatorul poate interpreta rezultatul cu mai multă prudență.

6.11.9 Reproducibilitate

Fluxul experimental este reproductibil prin comenzile din [Anexa A](#). Datele brute sunt salvate în JSONL, rapoartele de normalizare sunt salvate în JSON, iar antrenarea produce fișiere CSV cu predicțiile fiecărui model. Această structură permite refacerea graficelor și verificarea deciziilor de filtrare.

Limitarea practică este că scraping-ul live nu este perfect reproductibil. Anunțurile dispar, prețurile se schimbă, iar site-urile modifică structura paginilor. Din acest motiv, seturile JSONL salvate local sunt tratate ca snapshot-uri experimentale.

Capitolul 7

Concluzii și direcții viitoare

7.1 Dificultăți întâlnite

Dezvoltarea proiectului a fost mai dificilă în zona datelor decât în zona modelelor. Modelele pot fi înlocuite și comparate relativ ușor odată ce există un CSV curat. În schimb, obținerea unui set de date suficient de mare și suficient de coerent a necesitat multe iterații.

Pentru Autovit, principala dificultate a fost variabilitatea câmpurilor. Nu toate anunțurile includ aceleași date, iar unele informații sunt afișate în forme diferite. De exemplu, kilometrajul, puterea sau versiunea pot apărea ca text formatat, iar anumite câmpuri pot lipsi complet. În plus, anunțurile pot fi duplicate, promovate sau republicate.

Pentru mobile.de, dificultățile au fost mai severe. Site-ul folosește mecanisme de protecție care pot bloca accesul automat. În unele teste, cererile au returnat erori 401, 403 sau 429. În alte teste, pagina randată nu conținea datele așteptate sau afișa conținut intermediar. Integrarea linkurilor de detaliu a necesitat tratarea variantelor localizate, iar extragerea imaginii corecte a necesitat filtrarea imaginilor de dealer. O soluție experimentală care a funcționat pentru creșterea setului de date a fost citirea paginilor SEO publice printr-un format Markdown intermediar, urmată de parsare și deduplicare.

O dificultate separată a fost legată de predicțiile prea apropiate între mașini diferite. Aceasta a fost observată în testarea manuală a aplicației, când modele precum SVR sau KNN întorceau valori foarte asemănătoare pentru anunțuri cu prețuri reale mult diferite. Problema a indicat că pipeline-ul trebuia îmbunătățit prin date mai multe, transformare logaritmică, selecție de caracteristici și medie ponderată între modele.

Tabela 7.1: Dificultăți și soluții aplicate

Dificultate	Efect	Soluție aplicată
Structură HTML instabilă	Câmpuri lipsă sau extrase greșit	Parsare tolerantă și păstrarea datelor brute
Duplicate în pagini de căutare	Supra-reprezentarea unor anunțuri	Chei exacte și chei fuzzy
Blocări mobile.de	Dataset inițial mic și scraping nesigur	Headere de browser, fallback Playwright, snapshot prin pagini SEO Markdown
Imagini greșite mobile.de	Afișare banner dealer în locul mașinii	Prioritizarea galeriei vehiculului
Predictii prea uniforme	Verdict slab pentru mașini diferite	Log-preț, set mai mare, medie ponderată
Artefacte mari de model	Deployment greu pe GitHub și Render	Compresie joblib și folder de artefacte

7.2 Limitări

Prima limitare este dimensiunea și compoziția setului de date. Deși setul final are mii de anunțuri, el este încă dominat de Autovit. Pentru mobile.de, volumul a crescut semnificativ prin metoda Markdown, dar câmpurile sunt mai incomplete și nu justifică încă un model separat de încredere. O versiune viitoare ar trebui să colecteze date mobile.de prin acces oficial la API, căutări filtrate la intervale controlate sau un mecanism manual de export dacă scraping-ul automat rămâne blocat.

A doua limitare este lipsa unor caracteristici importante. Starea tehnică, istoricul accidentelor, dotările exacte, reviziile, numărul de proprietari și starea anvelopelor nu sunt disponibile consecvent. Aceste informații pot schimba mult prețul, dar nu pot fi deduse sigur din câmpurile actuale.

A treia limitare este faptul că modelele învață din prețuri cerute, nu din prețuri tranzacționate. Dacă vânzătorii publică frecvent prețuri negociabile, modelul va învăța piața anunțurilor, nu piața tranzacțiilor reale. Această limitare nu poate fi eliminată fără acces la date de vânzare efectivă.

A patra limitare este dependența aplicației de site-uri externe. Dacă Autovit sau mobile.de schimbă structura paginii, schimbă endpoint-urile sau blochează cererile cloud, predicția din link poate eșua. Pentru o aplicație publică, ar fi necesar un fallback prin formular manual, unde utilizatorul introduce marca, modelul, anul, kilometrajul și restul caracteristicilor.

7.3 Concluzii

Lucrarea a demonstrat că o aplicație web pentru evaluarea prețurilor autoturismelor second-hand poate fi construită pornind de la date publice colectate automat. Sistemul rezultat include scraping, normalizare, deduplicare, selecție statistică de caracteristici, antrenare de modele ML

și interfață web cu autentificare și istoric.

Rezultatele experimentale sunt încurajatoare. Pe setul combinat extins, cel mai bun model a obținut un scor R^2 de aproximativ 0.923, iar eroarea absolută medie a celui mai bun model după MAE a fost de aproximativ 2696 EUR. Aceste valori sunt potrivite pentru un prototip academic și pentru filtrare preliminară, dar nu justifică folosirea aplicației ca unic criteriu de cumpărare.

Un rezultat important al proiectului este faptul că aplicația nu ascunde complexitatea modelării. Utilizatorul vede estimările mai multor modele, poate schimba metoda de combinare și poate ajusta toleranța verdictului. Această transparență este utilă deoarece piața auto este zgomotoasă, iar un singur număr poate fi înșelător.

7.4 Direcții viitoare

Prima direcție viitoare este extinderea datasetului. Pentru acuratețe mai bună, este necesară colectarea de date pe mai multe filtre: marcă, model, interval de preț, an și combustibil. O singură căutare largă produce multe duplicate și nu acoperă uniform piața. În plus, aplicația ar putea accepta și alte platforme relevante pentru România, de exemplu OLX Auto, pentru a reduce dependența de două surse și pentru a acoperi mai bine anunțurile locale.

A doua direcție este îmbunătățirea suportului mobile.de. O soluție robustă ar putea combina scraping prudent, cache local, import manual de fișiere exportate și formulare de fallback. Într-un serviciu public, dependența de scraping live trebuie redusă.

A treia direcție este adăugarea unor modele moderne de boosting, precum XGBoost, LightGBM sau CatBoost. Aceste modele sunt des folosite pentru date tabulare și ar putea îmbunătăți rezultatele, mai ales dacă datasetul crește.

A patra direcție este explicabilitatea. Aplicația ar putea afișa factorii care au influențat cel mai mult estimarea: kilometraj mare, model premium, an recent, putere mare sau lipsa unor date importante. Această funcționalitate ar ajuta utilizatorul să înțeleagă verdictul, nu doar să îl accepte.

A cincea direcție este transformarea aplicației într-un serviciu complet deployat. Configurația Render există, dar o variantă stabilă ar trebui să folosească stocare persistentă pentru conturi, cache pentru predicții, monitorizare pentru erorile de scraping și o strategie mai bună pentru artefactele mari de model, precum Git LFS sau stocare externă. După extinderea datasetului, validarea rezultatelor pe un număr mai mare de utilizatori și consolidarea infrastructurii, aplicația ar putea fi monetizată prin abonamente, rapoarte detaliate sau servicii dedicate dealerilor, fără a compromite accesul la o evaluare de bază.

7.5 Încheiere

CarValuator arată că un proiect de data science aplicat nu se reduce la alegerea unui model. Datele, normalizarea, verificarea, interfața și limitele operaționale sunt la fel de importante. În

această lucrare, provocarea centrală nu a fost doar să prezicem un preț, ci să realizez un flux complet care pornește de la un link real și ajunge la o explicație utilă pentru utilizator.

Prin combinația dintre scraping, învățare automată și aplicație web, proiectul oferă o bază solidă pentru dezvoltări viitoare și pentru un eventual deployment la scară mai mare.

Anexa A

Comenzi utile pentru reproducerea experimentelor

Această anexă listează comenzile principale folosite în proiect. Ele sunt utile pentru refacerea fluxului experimental pe orice sistem cu Windows PowerShell.

A.1 Pornirea mediului

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 python -m pip install -e .
4 python -m playwright install chromium
```

Listing A.1: Inițializarea mediului Python

A.2 Colectarea datelor

```
1 python -m carvaluator_scraper.cli scrape-search autovit "https://www
  .autovit.ro/autoturisme" --pages 150 --delay 0.4 --output data\
  autovit_xl.jsonl
```

Listing A.2: Exemplu de scraping Autovit

```
1 python -m carvaluator_scraper.cli scrape-search mobilede "https://
  suchen.mobile.de/fahrzeuge/search.html?dam=false&isSearchRequest=
  true&ref=quickSearch&s=Car&vc=Car" --pages 5 --delay 1 --output
  data\mobilede_search.jsonl
```

Listing A.3: Exemplu de scraping mobile.de

```
1 python scripts\scrape_mobilede_reader.py --output data\  
    mobilede_reader_20260606.jsonl --report data\  
    mobilede_reader_20260606_report.json --cache-dir data\  
    mobilede_reader_cache_20260606
```

Listing A.4: Scraping experimental mobile.de prin pagini Markdown

A.3 Pregătirea setului de date

```
1 python -m carvaluator_scraper.cli export-csv data\autovit_xl.jsonl  
    --output data\autovit_xl.csv --drop-fuzzy-duplicates --report  
    data\autovit_xl_export_report.json
```

Listing A.5: Export CSV pentru antrenare

```
1 python -m carvaluator_scraper.cli export-csv data\  
    mobilede_reader_20260606.jsonl --output data\  
    mobilede_reader_20260606.csv --drop-fuzzy-duplicates --report  
    data\mobilede_reader_20260606_export_report.json
```

Listing A.6: Export CSV pentru setul mobile.de colectat prin reader

```
1 python -m carvaluator_scraper.cli export-csv data\autovit_xl.jsonl  
    data\autovit_more_20260526.jsonl data\  
    mobilede_consolidated_20260606.jsonl data\  
    mobilede_reader_20260606.jsonl --output data\  
    combined_reader_20260606.csv --drop-fuzzy-duplicates --report  
    data\combined_reader_20260606_report.json
```

Listing A.7: Export CSV combinat pentru experimentul final

A.4 Antrenarea modelelor

```
1 python -m carvaluator_scraper.cli train-models data\  
    combined_reader_20260606.csv --output-dir data\  
    model_results_combined_reader_20260606_log --log-target
```

Listing A.8: Antrenare cu țintă logaritmică

A.5 Pornirea aplicației

```
1 .\scripts\start-carvaluator.ps1 -Port 8017 -Background -StopExisting
```

Listing A.9: Pornire automată cu scriptul proiectului

```
1 $env:CARVALUATOR_MODEL_BUNDLE = "data\  
    model_results_combined_reader_20260606_log\best_model.joblib"  
2 $env:CARVALUATOR_SIMILARITY_CSV = "data\similarity_current.csv"  
3 $env:CARVALUATOR_API_PORT = "8017"  
4 .\venv\Scripts\python.exe -m carvaluator_scraper.api
```

Listing A.10: Pornirea serverului local

După pornire, aplicația rulează local la adresa `http://127.0.0.1:8017`. Ruta `/health` verifică dacă modelul, datasetul de similaritate și graficele principale sunt disponibile.

A.6 Reîmprospătarea sugestiilor similare

Anunțurile din baza de similaritate pot expira chiar dacă rândurile rămân valide pentru analiza statistică. Din acest motiv, lotul folosit de KNN trebuie actualizat periodic, separat de antrenarea modelelor. Comanda următoare colectează anunțuri Autovit curente, normalizează și deduplică rezultatul, arhivează fișierele cu timestamp, actualizează `data/similarity_current.csv` și repornește serverul:

```
1 .\scripts\refresh-similarity.ps1 -Pages 100 -RestartServer
```

Listing A.11: Actualizarea locală a lotului de sugestii

Actualizarea nu modifică estimarea de preț și nu reantrenează modelele. Ea schimbă numai mulțimea de candidați din care motorul KNN selectează anunțurile similare. Rapoartele deja salvate în istoricul conturilor păstrează sugestiile existente la momentul analizei.

Pentru ca același lot să fie inclus și în deployment-ul Render se folosește opțiunea suplimentară:

```
1 .\scripts\refresh-similarity.ps1 -Pages 100 -UpdateRenderArtifacts -  
    RestartServer
```

Listing A.12: Actualizarea sugestiilor locale și a artefactelor Render

Această comandă actualizează doar folderul local `deploy_artifacts`. Site-ul public se schimbă abia după commit, push și redeploy, deoarece instanța Render nu poate citi fișierele din folderul local `data`.

A.7 Pregătirea pentru Render

```
1 .\scripts\prepare-render-artifacts.ps1 -Clean
```

Listing A.13: Pregătirea artefactelor pentru deployment Render

Comanda creează folderul `deploy_artifacts`, copiază modelul `best_model.joblib`, datasetul combinat și graficele generate la antrenare. Fișierul `render.yaml` folosește următoarele căi:

```
1 CARVALUATOR_MODEL_BUNDLE=deploy_artifacts/model_results/best_model.  
  joblib  
2 CARVALUATOR_SIMILARITY_CSV=deploy_artifacts/datasets/  
  similarity_current.csv  
3 CARVALUATOR_TRUST_PROXY_HEADERS=1  
4 CARVALUATOR_RATE_LIMIT_ENABLED=1  
5 CARVALUATOR_RATE_LIMIT_GLOBAL_REQUESTS=60  
6 CARVALUATOR_RATE_LIMIT_GLOBAL_WINDOW_SECONDS=60  
7 CARVALUATOR_RATE_LIMIT_LOGIN_REQUESTS=10  
8 CARVALUATOR_RATE_LIMIT_LOGIN_WINDOW_SECONDS=900  
9 CARVALUATOR_RATE_LIMIT_REGISTER_REQUESTS=3  
10 CARVALUATOR_RATE_LIMIT_REGISTER_WINDOW_SECONDS=3600  
11 CARVALUATOR_RATE_LIMIT_PREDICT_REQUESTS=5  
12 CARVALUATOR_RATE_LIMIT_PREDICT_WINDOW_SECONDS=60
```

Listing A.14: Variabile principale pentru Render

Valorile de rate limiting corespund limitelor generale, de login, înregistrare și predicție. Variabila pentru antetele proxy trebuie activată numai în spatele proxy-ului Render; local poate rămâne dezactivată. Pentru deployment, scriptul produce un bundle redus de aproximativ 1 MB, care păstrează SVR, Ridge, KNN și Gradient Boosting. Modelul complet local, cu șapte pipeline-uri, are aproximativ 606 MB și nu este încărcat pe planul gratuit Render. Configurația curentă folosește Docker pentru a fixa versiunile scikit-learn și pentru a instala Chromium:

```
1 Dockerfile  
2 render.yaml  
3 deploy_artifacts/model_results/best_model.joblib  
4 deploy_artifacts/datasets/similarity_current.csv
```

Listing A.15: Fișierele folosite de Blueprint-ul Render

După publicarea modificărilor în GitHub se folosește opțiunea de ștergere a cache-ului urmată de redeploy. Ruta `/health` trebuie să confirme existența modelului și a datasetului, precum și versiunile scikit-learn 1.8.0 și Playwright 1.58.0. Reîmprospătarea lotului reduce probabilitatea linkurilor expirate, dar nu o poate elimina complet, deoarece un anunț se poate închide imediat după colectare.

Bibliografie

- [1] ACAROM. Înmatriculările de autoturisme în românia - martie 2024. <https://acarom.ro/inmatricularile-de-autoturisme-in-romania-martie-2024/>, 2024.
- [2] Autovit.ro. Evaluare autoturism: cum calculezi valoarea corectă a mașinii tale. <https://www.autovit.ro/blog/evaluare-autoturism-cum-calculezi-valoarea-corecta-masinii-tale/>, 2022.
- [3] Autovit.ro. Evaluare auto online - calculator evaluare gratuită a mașinii. <https://www.autovit.ro/evaluarea-masinii>, 2026.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [5] Harald Cramer. *Mathematical Methods of Statistics*. Princeton University Press, 1946.
- [6] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026.
- [7] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [8] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.
- [9] Jina AI. Jina reader. <https://r.jina.ai/>, 2026.
- [10] Vlad Krotov and Leiser Silva. Legality and ethics of web scraping. In *Twenty-fourth Americas Conference on Information Systems*, New Orleans, 2018.
- [11] Microsoft Playwright. Playwright python documentation. <https://playwright.dev/python/docs/intro>, 2026.
- [12] mobile.de. Insights api documentation. <https://services.sandbox.mobile.de/docs/insights-api.html>, 2026.
- [13] mobile.de. mobile.de search api documentation. <https://services.mobile.de/docs/search-api.html>, 2026.
- [14] mobile.de. Seller api documentation. <https://services.mobile.de/docs/seller-api.html>, 2026.
- [15] Karl Pearson. Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58:240–242, 1895.
- [16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg,

- Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [17] Render. Render web services documentation. <https://render.com/docs/web-services>, 2026.
- [18] Revista Biz. Creștere a pieței auto în 2024. cele mai vândute mărci de mașini noi și second hand. <https://www.revistabiz.ro/crestere-a-pieteii-auto-in-2024-cele-mai-vandute-marci-de-masini-noi-si-second-hand/>, 2025.
- [19] Sherwin Rosen. Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55, 1974.
- [20] scikit-learn. Metrics and scoring documentation. https://scikit-learn.org/stable/modules/model_evaluation.html, 2026.
- [21] Colin Shearer. The crisp-dm model: The new blueprint for data mining. *Journal of Data Warehousing*, 5(4):13–22, 2000.
- [22] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004.
- [23] Nikiforos Zacharof, Jamil Nur, Dimitrios Kourtesis, Jette Krause, and Georgios Fontaras. A review of the used car market in the european union. Technical report, European Commission, Joint Research Centre, 2025.